

Django

Django Application:

- An app in Django is a sub-module of a project, and it is used to implement some functionality(business logic).
- We can create multiple apps within a single Django project. And these apps can be independent of one another.

Note: The difference between a project and app is, a project is a collection of configuration files and apps whereas the app is a web application which is written to perform business logic.

Benefits of using Django apps –

- Django apps are reusable i.e. a Django app can be used with multiple projects.
- loosely coupled i.e. almost independent components
- Multiple developers can work on different components.
- Debugging and code organization is easy. Django has an excellent debugger tool.
- It has in-built features like admin pages, etc, which reduces the effort of building the same from scratch.

App creation in Django :

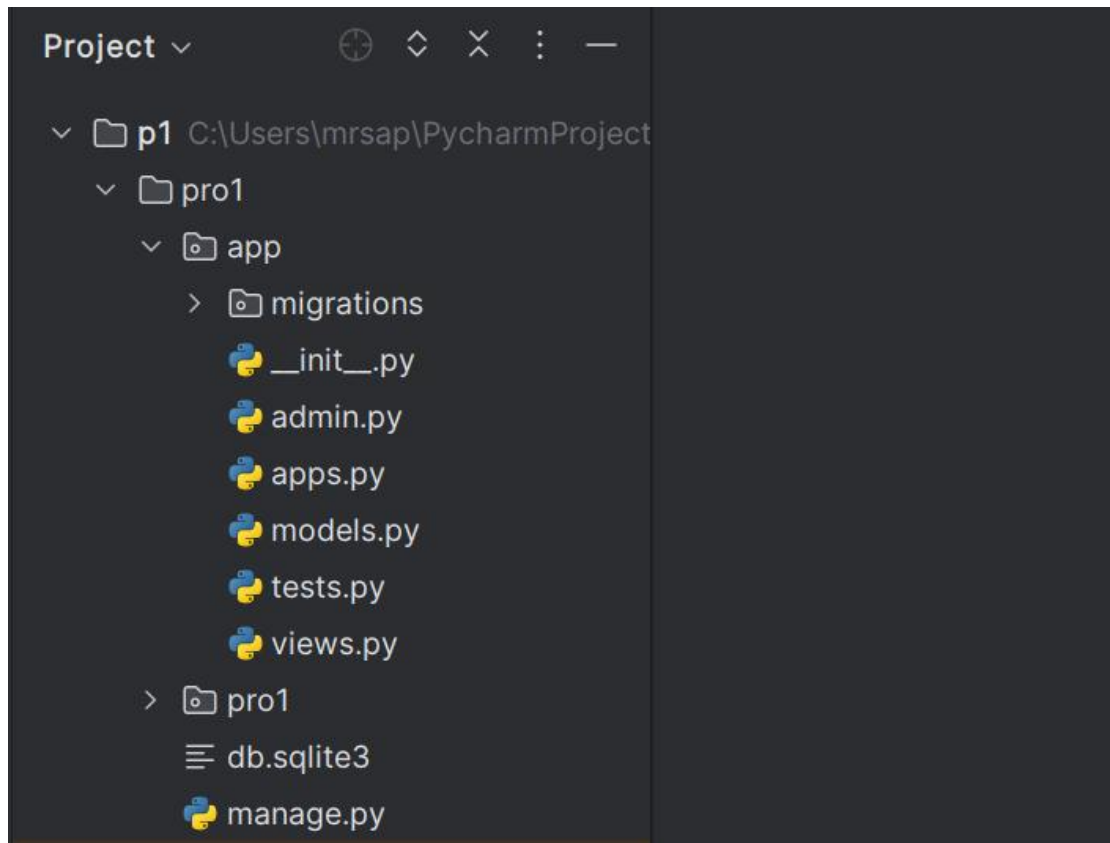
Step 1:- **Create django app**

To create a basic app in your Django project you need to go to the directory containing manage.py and from there enter the command.

python manage.py startapp projectApp
Django-admin startapp appname

```
(venv) C:\Users\mrsap\PycharmProjects\p1\pro1>django-admin startapp app
```

Folder Structure:



- **`__init__.py`** – a special Python file that is used to indicate that the directory it is present in is a Python package.
- **`admin.py`** – This file is used to display your models in the Django admin panel. You can also customize your admin panel.
- **`models.py`** – This is where all the application models are stored.
- **`tests.py`** – This file is use for unit tests
- **`views.py`** – This file contain view.

What is view?

A view is a place where we put our business logic of the application. View are Python functions or classes that receive a web request and return as a web response. The response can be a simple HTTP response, an HTML template response, or an HTTP redirect response that redirects a user to another page. Views hold the logic that is required to return information as a response in whatever form to the user.

There are 2 types of view in Django

- 1.Function Base View
- 2.Class Base View

1. Function-Based Views

Function-based views are good for beginners. It is very easy to understand in comparison to class-based views. function-based views are easy to understand but due to the code redundancy in a large Django project, you will find similar kinds of functions in the views.

Pros:

- Easy to read, understand and implement.
- Explicit code flow
- Straightforward usage of decorators.
- Good for the specialized functionality.

Cons:

- Code redundancy and hard to extend.
- Conditional branching will be used to handle HTTP methods.

Step 2:- Define app level View

Inside views.py --> PROJNAME-->APPNAME-->views.py

```
from django.http import HttpResponseRedirect

# Create your views here.

def view1(request):
    return HttpResponseRedirect("This is My 1st View")

def view2(request):
    return HttpResponseRedirect("This is My 2nd View")
```

- First, we import the class HttpResponseRedirect from the django.http module.
- Next, we define a function called view1. Each view function takes an HttpRequest object as its first parameter, which is typically named request.
- The view returns an HttpResponseRedirect object that contains the generated response. Each view function is responsible for returning an HttpResponseRedirect object.

2. Class-Based View

- Class-based views are the alternatives of function-based views.
- Class-based views don't replace function-based views, but they do have certain advantages over function-based views.

Pros:

- The most significant advantage of the class-based view is inheritance. In the class-based view, you can inherit another class, and it can be modified for the different use cases.
- Code reusability is possible in class-based views.
- Another advantage of using a class-based view is code structuring. In class-based views, you can use different class instance methods (eg. get, post) (instead of conditional branching statements inside function-based views) to generate different HTTP requests.

Cons:

- Complex to implement and harder to read

```
from django.http import HttpResponse
from django.views import View

class MyCreateView(View):

    def get(self, request):
        return HttpResponse("My first Class based view")
```

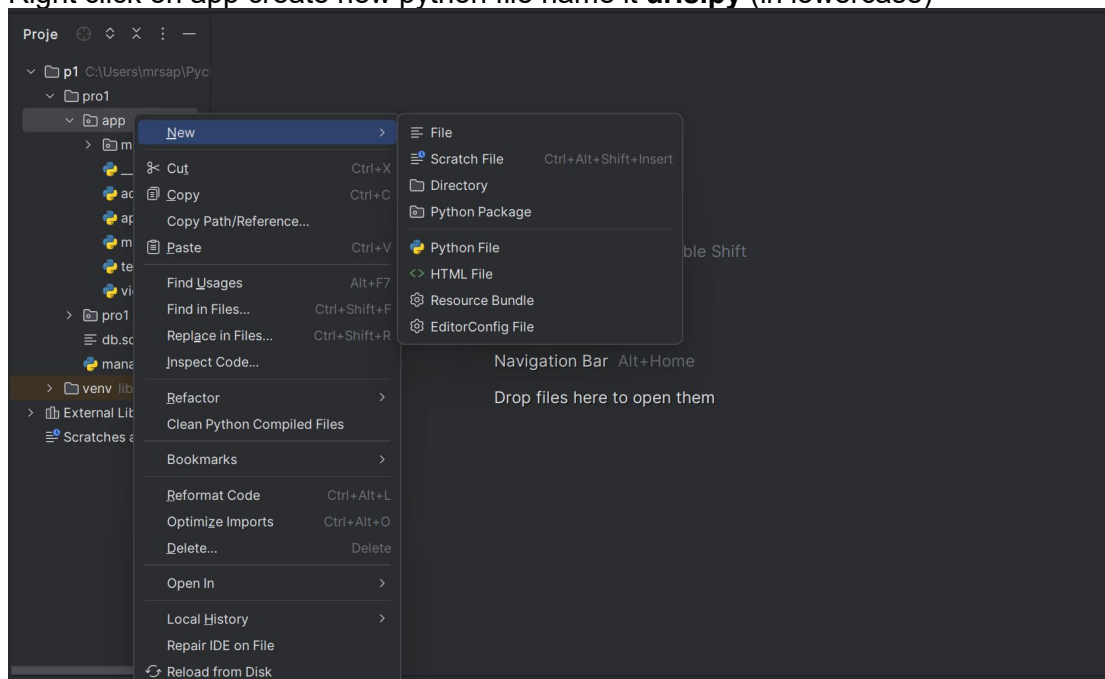
What is url?

- In Django, views are Python functions which take a URL request as parameter and return an HTTP response or throw an exception like 404. Each view needs to be mapped to a corresponding URL pattern.
- Django runs through each URL pattern, in order, and stops at the first one that matches the requested URL, matching against path_info.

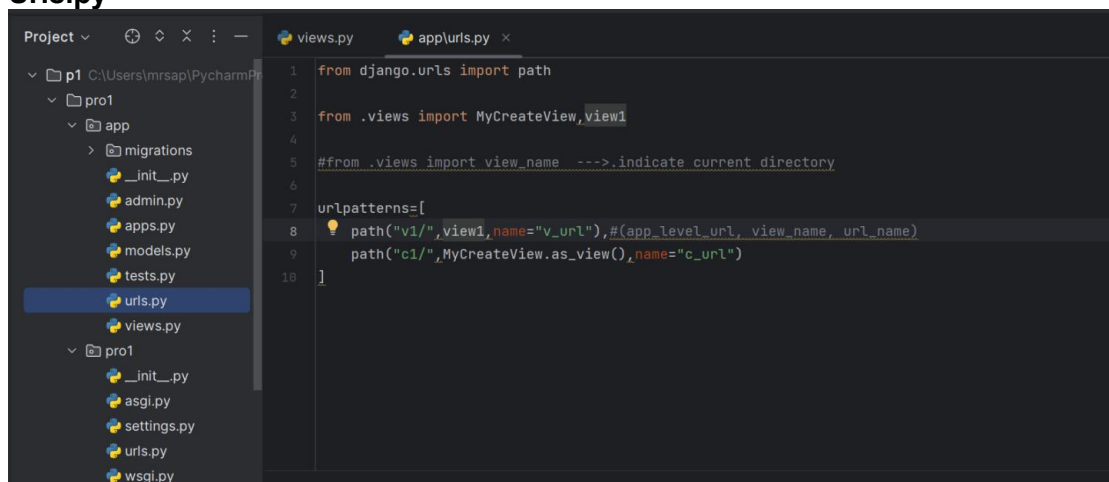
Step 3:- Define App_level url

And we have to explicitly create urls.py in our app.

Right click on app create new python file name it **urls.py** (in lowercase)



Create new urls.py → PROJNAME → APPNAME → urls.py (AppLevel urls) Urls.py



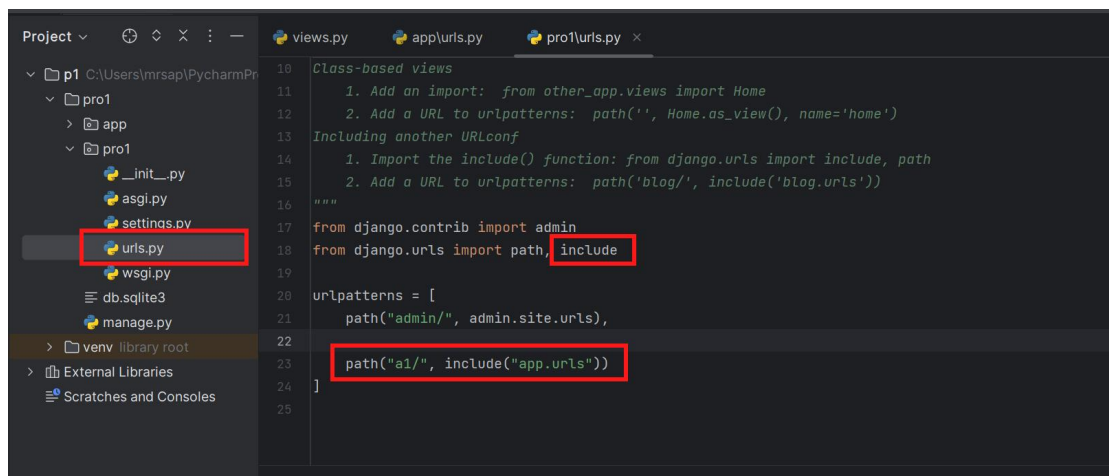
Note: First import View class from Django.views module.
Name of list must be **urlpatterns** (in lowercase).
we are passing 3 arguments here (app_level_url, view_name, url_name)
in case of class based views we have to add suffix(**as_view()**)
className.as_view().

Step 4:- Register app_url into project_url

Define an urlpattern(path) --> PROJNAME-->PROJNAME-->urls.py

Steps:-

- Let's open the file **urls.py** of the project. There is default file named urls.py in project.
- we import the **include** function from the **django.urls** and map the path of the **a1** to the **app.urls**.



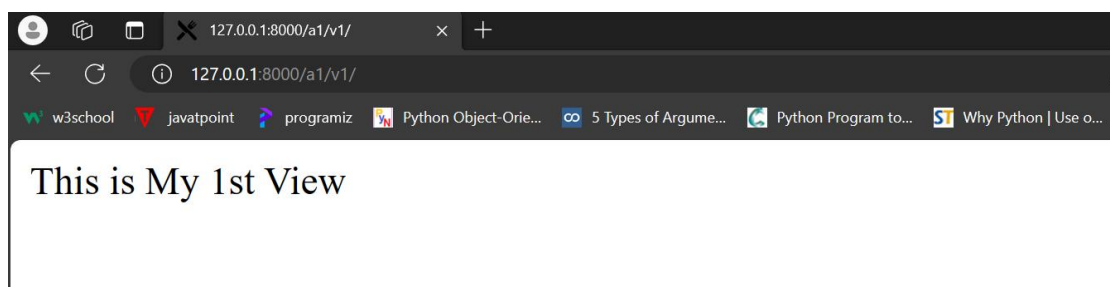
Step 5:- **Run Server**

Py manage.py runserver

Now, start the server and enter **localhost:8000/a1/v1** to the browser. This URL will be mapped into the list of URLs and then call the corresponding function from the views file.

In this example, v1 will be mapped and call view1 function from the views file. It is called **URL mapping**.

- First, the web browser sends an HTTP request to the URL `http://127.0.0.1:8000/a1/`
- Second, Django executes the `urls.py` in the `django_project` directory. It matches the `a1/` with the URL in the `urlpatterns` list in the `urls.py`. As a result, it sends " to the `urls.py` of the app application.
- Third, Django runs the `urls.py` file in the app application. It matches the " URL with the `views.view1` function and execute it, which returns an HTTP response that outputs a `h1` tag.
- Finally, Django returns a webpage to the web browser.

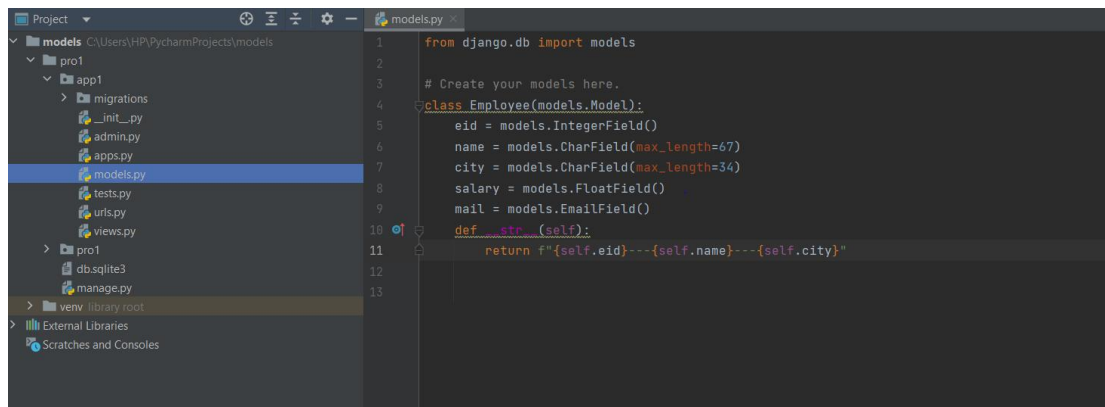


ORM(Object Relational Mapping)

The Django web framework includes a default object-relational mapping layer (ORM) that can be used to interact with data from various relational databases such as SQLite, PostgreSQL, and MySQL. Django allows us to add, delete, modify, and query objects, using an API called ORM. ORM stands for Object Relational Mapping. An object-relational mapper provides an object-oriented layer between relational databases and object-oriented programming languages without having to write SQL queries.

Step 1:- Defining Models

You create Python classes that subclass **django.db.models.Model**. Each class represents a database table, and attributes represent fields in the table.



Step 2:- Creating Migrations

Django's migration system allows you to create, apply, and manage changes to your models and database schema. After defining a model, you create a migration file that captures these changes:

python manage.py makemigrations

python manage.py migrate

Step 3:- Create interactive console

By using the shell, we can easily test and make changes to a project's models.

Py manage.py shell

```
(venv) C:\Users\HP\PycharmProjects\models\pro1>py manage.py shell
Python 3.10.7 (tags/v3.10.7:6cc6b13, Sep 5 2022, 14:08:36) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> 
```

Step 4:- To import model from the database

from APPNAME.models import MODELCLASSNAME

```
>>> from app1.models import Employee
```

Querying the Database:

You can perform database operations using the model classes and methods provided by Django's ORM.

1. How to get all records from table(Model)

We have a model called **Employee**. To get all records from model, we will use the

Employee.objects.all().

```
>>> Employee.objects.all()
<QuerySet [(<Employee: 1---ajay'---pune>, <Employee: 2---abhi---mumbai>, <Employee: 3---poonam---delhi>)]>
```

2. Add new Employee record in Employee Model:

Create a new instance instance = YourModel(name='John', age=25)
instance.save()

```
>>> e1 = Employee(eid=4,name="suraj",city="nagpur",salary=50000,mail="anu@gmail.com")
>>> e1.save()
>>> Employee.objects.all()
<QuerySet [(<Employee: 1---ajay'---pune>, <Employee: 2---abhi---mumbai>, <Employee: 3---poonam---delhi>, <Employee: 4---suraj---nagpur>)]>
```


3 Fetch particular record

```
>>> Employee.objects.get(eid=2)
<Employee: 2---abhi---mumbai>
```

4 Fetch all record depend on condition

YourModel.objects.filter(condition)

```
>>> Employee.objects.all()
<QuerySet [(<Employee: 1---ajay---pune>, <Employee: 2---abhi---mumbai>, <Employee: 3---poonam---delhi>, <Employee: 4---suraj---nagpur>, <Employee: 2---suraj---nagpur>, <Employee: 4---a
nuj---nagpur>)]>
>>> Employee.objects.filter(eid=2)
<QuerySet [(<Employee: 2---abhi---mumbai>, <Employee: 2---suraj---nagpur>)]>
>>>
```

5. Filtering records where salary is greater than 400000

Querying results = **YourModel.objects.filter(salary__gt=400000)**

```
>>> Employee.objects.all()
<QuerySet [(<Employee: name:anu city:pune salary:500000.0>, <Employee: name:Apurva city:pune salary:700000.0>, <Employee: name:Suraj Chauhan city:Mzr salary:400000.0>)]>
>>> Employee.objects.filter(salary__gt=400000)
<QuerySet [(<Employee: name:anu city:pune salary:500000.0>, <Employee: name:Apurva city:pune salary:700000.0>)]>
>>>
```

6.Delete particular Record

Delete instance = **YourModel.objects.get(name='John') instance.delete()**

```
>>> Employee.objects.all()
<QuerySet [(<Employee: name:Apurva city:pune salary:700000.0>, <Employee: name:Suraj Chauhan city:Mzr salary:400000.0>)]>
>>> Employee.objects.filter(eid=2).delete()
(1, {'app1.Employee': 1})
>>> Employee.objects.all()
<QuerySet [(<Employee: name:Suraj Chauhan city:Mzr salary:400000.0>)]>
>>>
```

7.Update Particular Record

```
>>> obj = Employee.objects.get(id=3)
>>> obj.name = 'dhiraj'
>>> obj.save()
>>> obj
<Employee: name:dhiraj city:Mzr salary:400000.0>
```

```
Update instance = YourModel.objects.get(fetch record)
instance.name = "update name"
instance.save()
```

The Django ORM abstracts away many complexities of database interaction, allowing you to focus on your application's logic. It supports various databases and provides a high level of flexibility in handling database operations.

Querys

1. objs = MODEL.objects.all()
2. obj = MODEL.objects.get(col=val) -->OBJ
3. objs = MODEL.objects.filter(col=val) -->QS
4. objs = MODEL.objects.exclude(col=val)
5. objs = MODEL.objects.filter(col__lt = val)
6. objs = MODEL.objects.filter(col__gt = val)
7. objs = MODEL.objects.filter(col__lte = val)
8. objs = MODEL.objects.filter(col__gte = val)
9. objs = MODEL.objects.filter(col__startswith = 'val')
10. objs = MODEL.objects.filter(col__endswith = 'val')
11. objs = MODEL.objects.filter(col__contains = 'val')
12. objs = MODEL.objects.order_by('col')

1. **Manipulating Data:** You can update, delete, or retrieve data using ORM methods:

```
#Update
13. obj = MODEL.objects.get(col=val)
obj.COL = NEWVAL
obj.save()
#Delete
14. obj = MODEL.objects.get(col=val)
obj.delete()
#Insert
15. obj = MODEL(col1=val,col2=val,col3=val..)
obj.save()
OR
obj = MODEL()
obj.col1 = val
obj.col2 = val
obj.col3 = val
```

```
obj.save()
```

Aggregation Funs

```
from django.db.models import Max,Min,Avg,Sum,Count
```

```
16. mx = MODEL.objects.all().aggregate(Max('col'))
```

```
17. mi = MODEL.objects.all().aggregate(Min('col'))
```

```
18. sm = MODEL.objects.all().aggregate(Sum('col'))
```

```
19. av = MODEL.objects.all().aggregate(Avg('col'))
```

```
20. ct = MODEL.objects.all().aggregate(Count('col'))
```

```
#And
```

```
21. objs = MODEL.objects.filter(condition1 , condition2)
```

```
#Or
```

```
22. objs = MODEL.objects.filter(condition1) | MODEL.objects.filter(condition2)
```

```
from django.db.models import Q
```

```
23. objs = MODEL.objects.filter(Q(condition1) | Q(condition2))
```

```
a]Student.objects.filter(marks=Student.objects.all().aggregate(mm=Max('marks'))['mm']).first()
```

```
b]max_marks = Student.objects.all().aggregate(mm=Max('marks'))['mm']
```

```
Student.objects.filter(marks=max_marks).first()
```

Models in Django

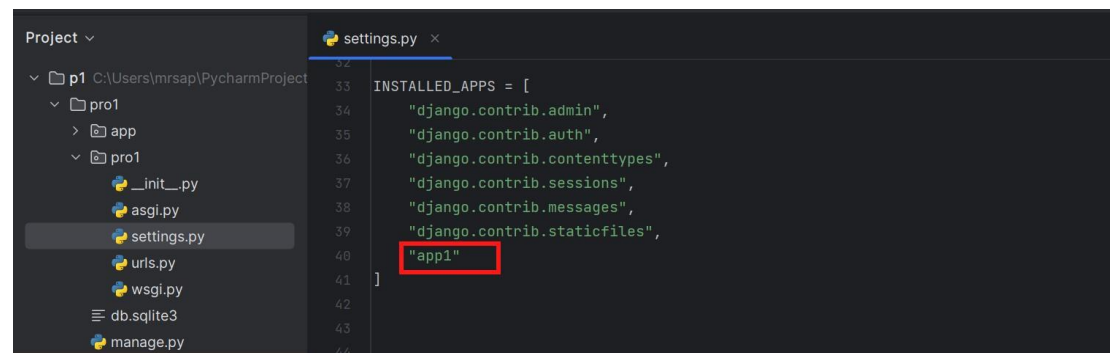
Models in Django

- In Django, A model is a class that represents table or collection in our DB, and where every attribute of the class is a field of the table or collection. Models are defined in the app/models.py (in our example: myapp/models.py).
- Each model class maps to a single table in the database.

- Django provides a big in-built support for database operations. Django provides one inbuilt database **sqlite3**.
- Django can provide support for other databases also like oracle,mysql, postgresql, etc.
- Model is defined in Models.py file. This file can contain multiple models.

Step 1:

you need to tell Django that you're going to use them by registering the application name in the **INSTALLED_APPS** list in the **settings.py** of the project:(Register app into the INSTALLED_APPS inside settings.py file.



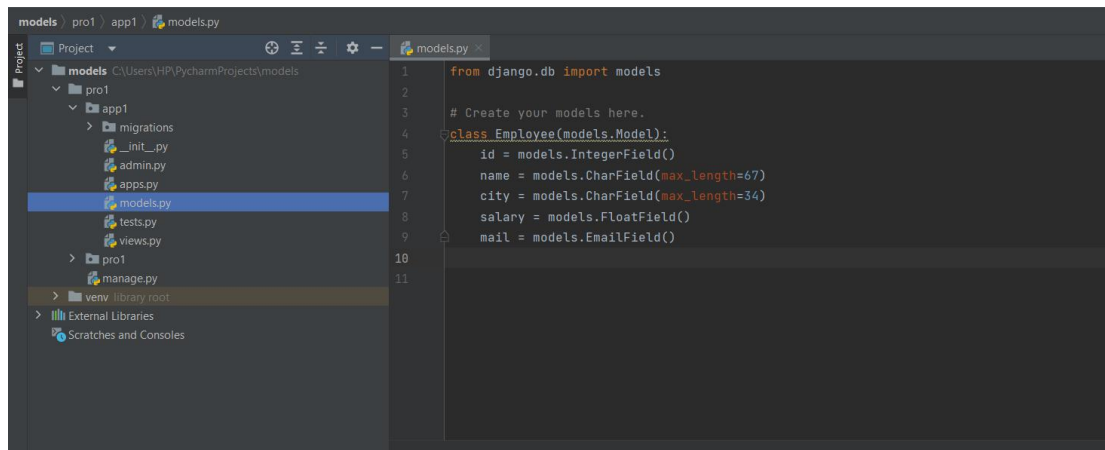
Step 2: Define Models

Open models.py file, Inside your app directory, locate the models.py file and define your models using Python classes. The fields are specified as class attributes and each attribute maps to a database column.

<ProName> →AppName →models.py file

Fields and Options:

- **Field Types:** Django provides various field types (CharField, IntegerField, DateTimeField, ForeignKey, etc.) that define the type of data each field will hold.
- **Field Options:** You can set various options for each field, like max_length for CharField or null and blank for allowing null values or empty strings.
- **Define Relationships (if needed):** If your model has relationships with other models (one-to-one, one-to-many, many-to-many), you can define them using ForeignKey, OneToOneField, or ManyToManyField.



we are creating a model Employee which has Five fields id, name, city, mail and salary.

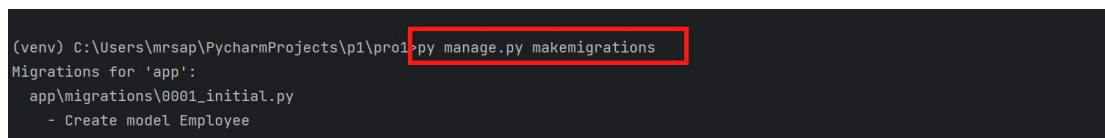
Model Class:

- A Model is a Python class which contains database information.
- A Model is a single, definitive source of information about our data. It contains fields and behavior of the data what we are storing.
- Each model maps to one database table.
- Every model is a Python class which is the child class of (django.db.models.Model)
- Each attribute of the model represents a database field.
- We have to write all model classes inside '**models.py**' file.

Step 3: Make Migrations

After defining your models, generate Django migrations using the following command:

py manage.py makemigrations



The makemigrations in django the command is used to create database migration files based on the changes you've made to your models.(python code gets converted into SQL)

what is a migration file?

A migration file contains Python code that represents the changes to the database schema, such as creating new tables, altering existing tables, or adding new fields and you can see the created file after running the command.

When you make changes to your models, such as adding new fields, changing field types, or even creating new models, running makemigrations analyzes these changes and generates migration files that capture the changes in a human-readable format. These migration files are stored in your app's migrations directory.

step 4: Apply Migrations:

Apply the generated migrations to create or update your database schema:

py manage.py migrate

Execute the SQL code

```
C:\Users\mrsap\PycharmProjects\p2\pro1>py manage.py makemigrations
No changes detected

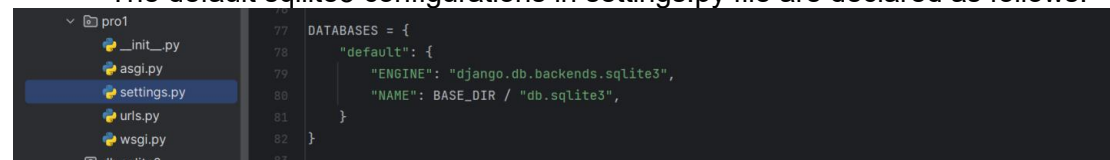
C:\Users\mrsap\PycharmProjects\p2\pro1>py manage.py migrate
Applying auth.0008_alter_user_username_max_length... OK
Applying auth.0009_alter_user_last_name_max_length... OK
Applying auth.0010_alter_group_name_max_length... OK
Applying auth.0011_update_proxy_permissions... OK
Applying auth.0012_alter_user_first_name_max_length... OK
Applying sessions.0001_initial... OK
```

- This command creates tables, modifies columns, adds indexes, and performs any other database-related operations needed to reflect the changes you've made.
- The **migrate** command takes care of the order in which migrations are applied, ensuring that dependencies between migrations are satisfied.

Once your models are defined and migrations are applied, you can use these models in your views, serializers, forms, etc., to interact with the database.

Database Configuration:

- Django by **default provides sqlite3 database**. If we want to use this database, we are not required to do any configurations.
- The default sqlite3 configurations in settings.py file are declared as follows.



```
77 DATABASES = {
78     "default": {
79         "ENGINE": "django.db.backends.sqlite3",
80         "NAME": BASE_DIR / "db.sqlite3",
81     }
82 }
83
```

If we don't want sqlite3 database then we have to configure our own database with the following parameters.

- 1) ENGINE: Name of Database engine
- 2) NAME: Database Name
- 3) USER: Database Login user name
- 4) PASSWORD: Database Login password
- 5) HOST: The Machine on which database server is running

6) PORT: The port number on which database server is running.

Configuration of MySQL Database:

- First we have to create our own logical database in the MySQL.

<mysql> create database <dbname>;

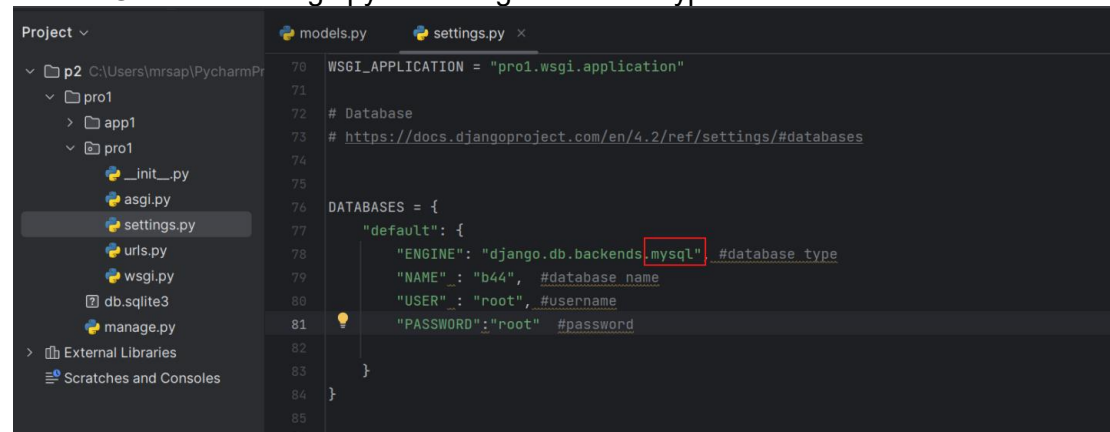
- We have to install mysqlclient using pip :

pip install mysqlclient

```
C:\Users\mrsap\PycharmProjects\p2\pro1>pip install mysqlclient
Requirement already satisfied: mysqlclient in c:\users\mrsap\appdata\local\programs\python\python311\lib\site-packages (2.1.1)

[notice] A new release of pip is available: 23.1.2 -> 23.3.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

- Go to the settings.py file change database type.



Admin Panel

Admin Panel:

The Django admin application can use your models to automatically build a site area that you can use to create, view, update, and delete records. This can save you a lot of time during development, making it very easy to test your models and get a feel for whether you have the right data.

Creating an admin panel in Django is quite straightforward and powerful due to Django's built-in admin interface.

Steps to Create an Admin Panel in Django:

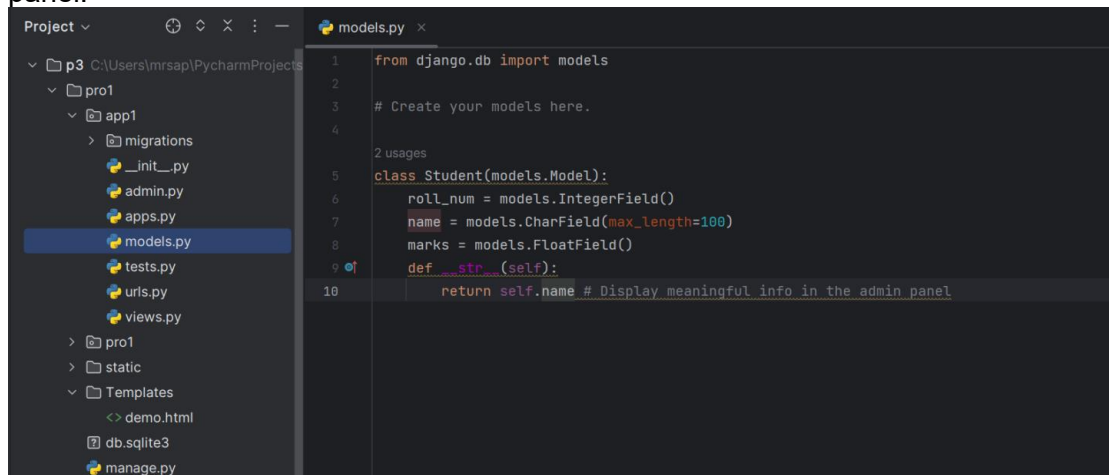
Step 1:- Create a Django App

If you haven't already, create a Django app using the command line:

django-admin startapp yourappname

Step 2:- Define Models

In your app's **models.py** file, define the models you want to manage via the admin panel:



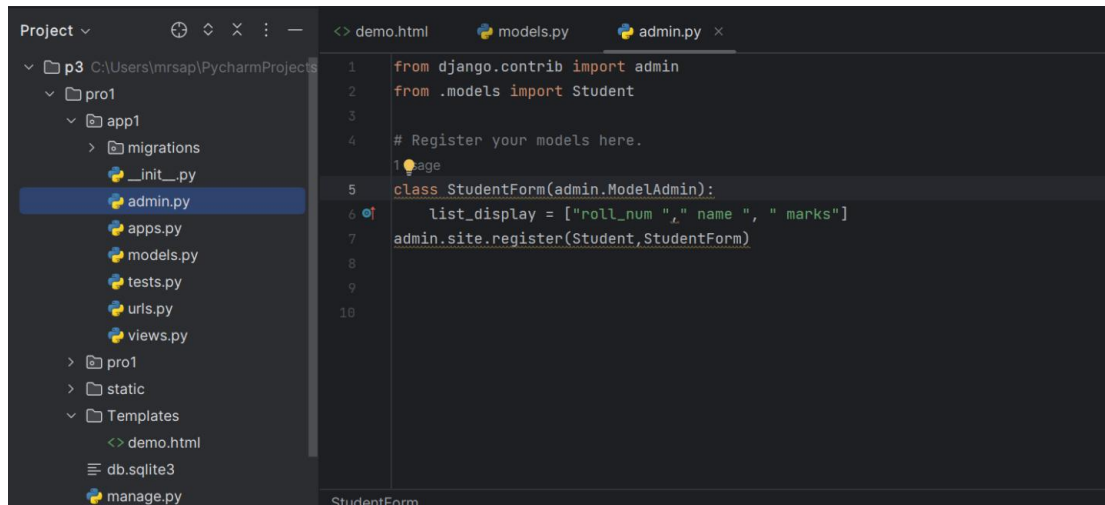
```
1 from django.db import models
2
3 # Create your models here.
4
5 2 usages
6 class Student(models.Model):
7     roll_num = models.IntegerField()
8     name = models.CharField(max_length=100)
9     marks = models.FloatField()
10     def __str__(self):
11         return self.name # Display meaningful info in the admin panel
```

Enter command in Terminal:

py manage.py makemigrations
py manage.py migrate

Step3: - Register Models in Admin Panel

In the app's **admin.py** file, register your models to make them accessible in the admin panel:



list_display to control which fields are displayed on the change list page of the admin. If you don't set **list_display**, the admin site will display a single column that displays the `__str__()` representation of each object.

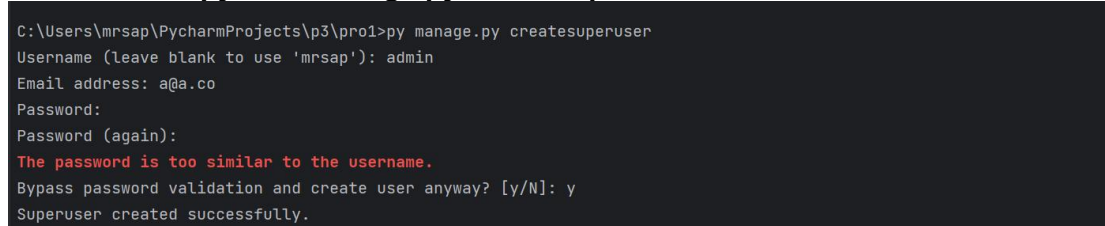
The Django admin is an automatically-generated user interface for Django models. The **register function** is used to add models to the Django admin so that data for those models can be created, deleted, updated and queried through the user interface.

```
@admin.register(model_nm)
admin.site.register(model_nm, admin_nm)
```

Step 4:- Create Superuser

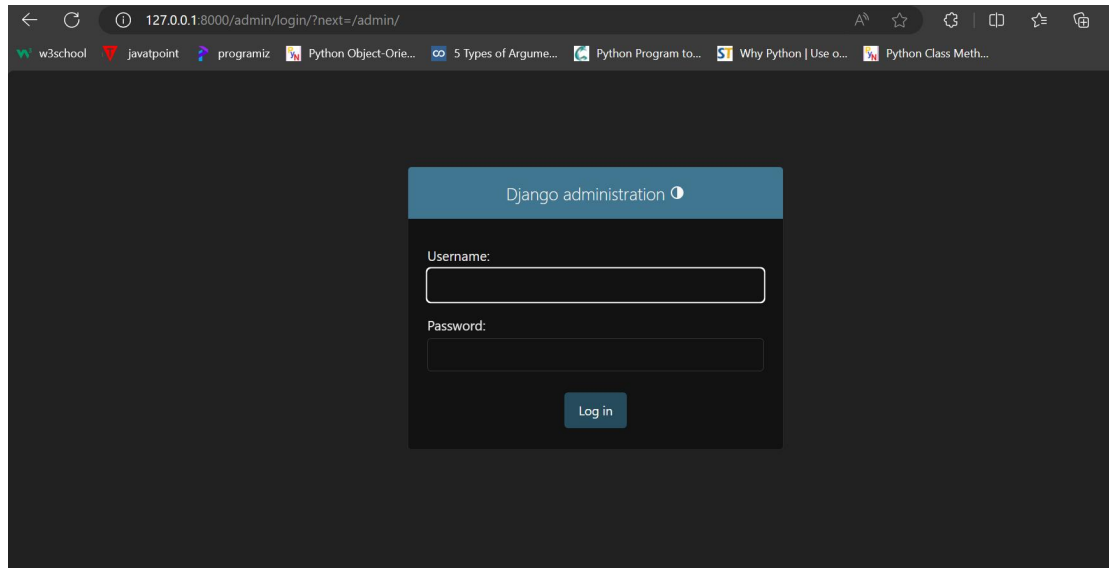
Run the following command to create a superuser who can access the admin panel:

python manage.py createsuperuser

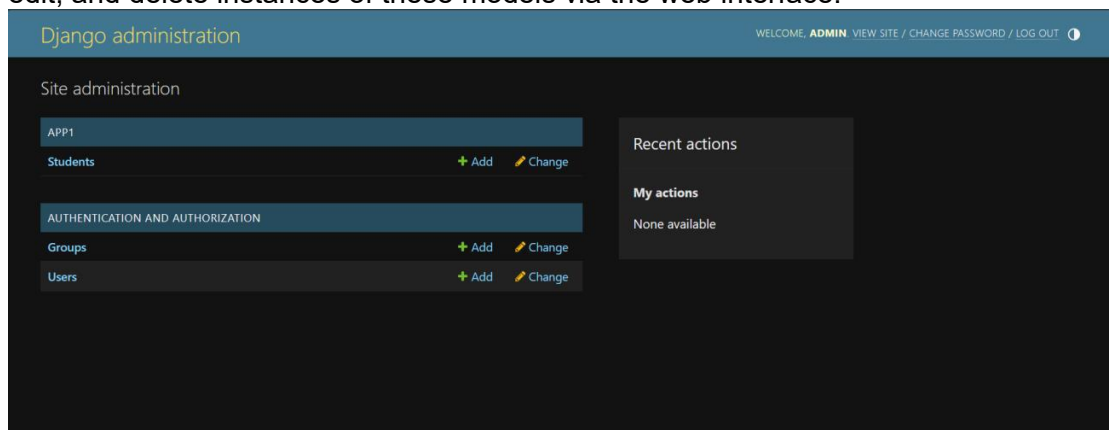


Step 5:- Start Development Server
`python manage.py runserver`

Step 6:- Access Admin Panel
Go to **`http://127.0.0.1:8000/admin/`** in your browser and log in using the superuser credentials created earlier.



You should now see the registered model(s) listed in the admin panel. You can add, edit, and delete instances of these models via the web interface.



Django Forms

Django Forms

Django forms are an advanced set of HTML forms that can be created using python and support all features of HTML forms in a pythonic way. This post revolves around how to create a basic form using various Form Fields and attributes. Creating a form in Django is completely similar to creating a model, one needs to specify what fields would exist in the form and of what type. For example, to input, a registration form one might need First Name (CharField), Roll Number (IntegerField), and so on.

Types of Forms

- Raw HTML Form
- Django Form class
- Django ModelForm

1.Raw HTML Form

In Django, you can create HTML forms in a variety of ways, including using raw HTML in your templates or by using Django's form handling and rendering capabilities. To create a form using raw HTML in a Django template, follow these steps:

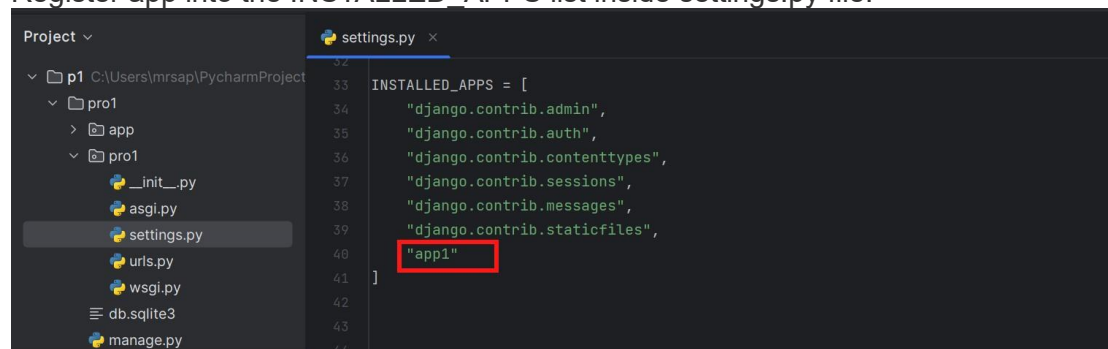
Step 1: Create a Django App

If you haven't already, create a Django app using the command line:

```
python manage.py startapp myapp
```

Step 2: Register the application

Register app into the `INSTALLED_APPS` list inside `settings.py` file.



Step 3: Create a Django Template:

Create or open an existing Django template directory (.html file) where you want to include the raw HTML form.

```
<> index.html x
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Title</title>
6  </head>
7  <body>
8      <form method="post">
9          {% csrf_token %}
10         <!-- csrf_token is used to prevent Cross-Site Request Forgery attacks -->
11
12         <label >Username:</label>
13         <input type="text" name="un" >
14
15         <label >Password:</label>
16         <input type="password" name="pwd">
17
18         <input type="submit" value="Submit">
19     </form>
20 </body>
21 </html>
22
```

action: This attribute specifies where the form data should be sent when submitted. Replace "/submit-form" with the URL or path where your form data should be processed.

method: This attribute specifies the HTTP method to be used when submitting the form. POST is commonly used to send data securely.

Inside the <form> tags, you can add various input fields using the <input> element:

Explanation:

- <label>: Provides a label for an input field. The for attribute links the label to its corresponding input field.
- <input>: Creates different types of input fields (text, email, textarea, submit, etc.). Each has different attributes like type, id, and name.
- <input type="submit">: Creates a submit button to send the form data.
- Closing the Form, Always ensure to properly close the form tag:</form>

Once the form is created, you can test it by opening the HTML file in a web browser. Fill in the form fields and submit it. The form data will be sent to the specified action URL using the specified method.

Note:

- Use appropriate **id** and **name** attributes for each **input field**.
- Always ensure proper security measures when handling form submissions to prevent exploits like SQL injection or cross-site scripting (XSS) attacks.
- Remember to include **{% csrf_token %}** inside the <form> tag when using the **POST** method to prevent CSRF (Cross-Site Request Forgery) attacks.

- In a Django view, you can access this data using `request.GET` for **GET** requests or `request.POST` for **POST** requests to retrieve the values by their respective name attributes specified in the form.

Methods:

GET Method:

- Sends data in the URL's query string.
- Parameters are visible in the URL.
- Limited data can be sent (URL has a length limit).
- Can be bookmarked and cached.
- Generally used for retrieving data from the server.
- Not secure for sensitive information as data is visible in the URL.

Example:

```
<form method="GET">
<input type="text" name="username">
<input type="submit" value="Submit">
</form>
```

If submitted, the URL might look like: <process.php?username=enteredName>.

POST Method:

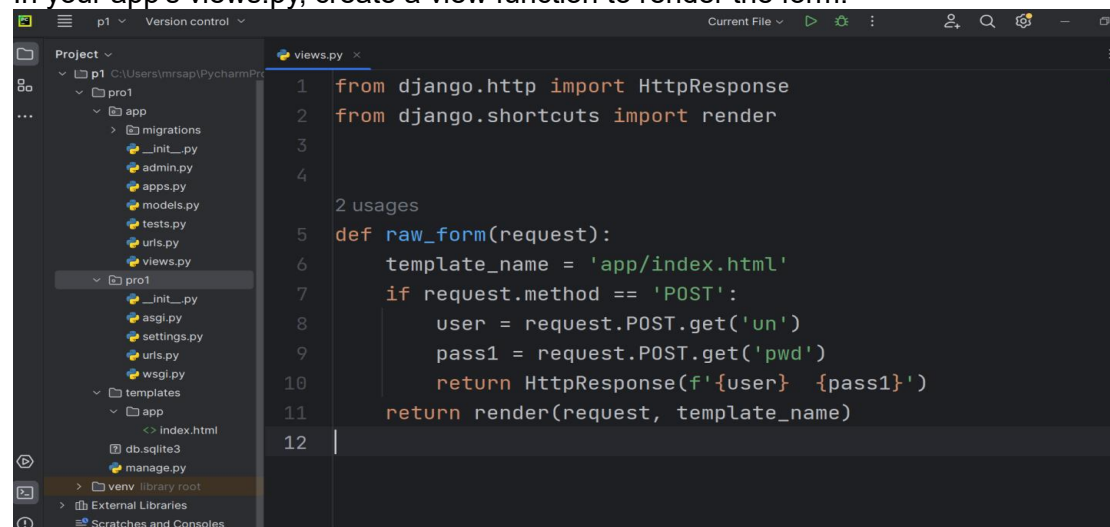
- Sends data in the body of the HTTP request.
- Parameters are not visible in the URL.
- Can send larger amounts of data compared to GET.
- Not bookmarked or cached (by default).
- More secure for sensitive information as data is not visible in the URL.

Example:

```
<form method="POST">
<input type="text" name="username">
<input type="submit" value="Submit">
</form>
```

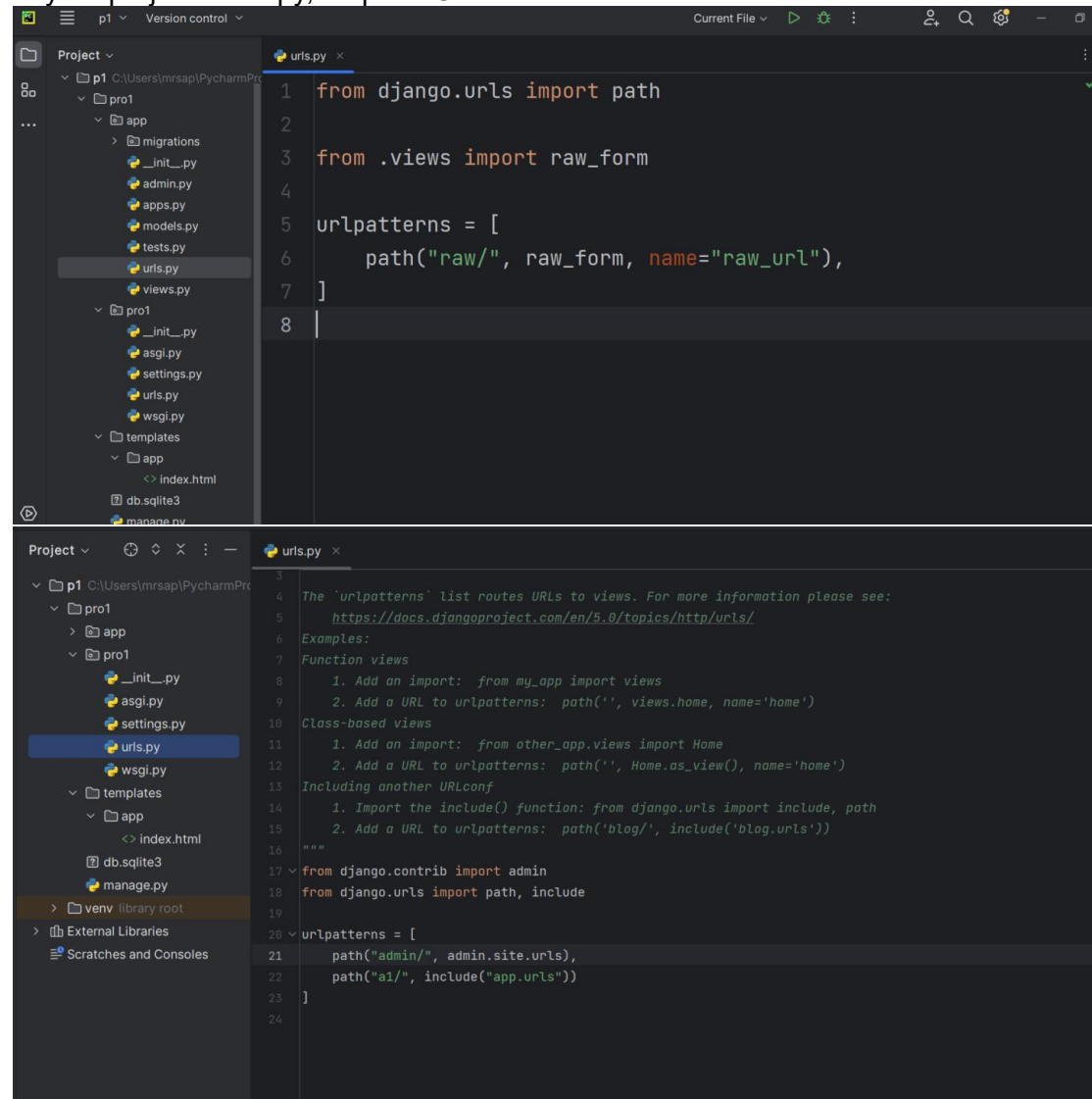
Step 4: Create a View

In your app's `views.py`, create a view function to render the form:



Step 5: Configure URLs

In your project's `urls.py`, map the URL to the view:



The first screenshot shows the `urls.py` file in a PyCharm IDE. The file contains the following code:

```
1 from django.urls import path
2
3 from .views import raw_form
4
5 urlpatterns = [
6     path("raw/", raw_form, name="raw_url"),
7 ]
8
```

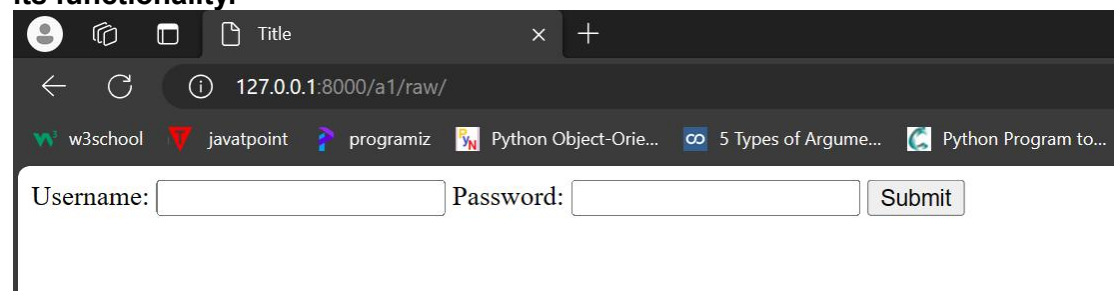
The second screenshot shows the same `urls.py` file with additional comments and imports. The code is as follows:

```
3
4 The 'urlpatterns' list routes URLs to views. For more information please see:
5     https://docs.djangoproject.com/en/5.0/topics/http/urls/
6 Examples:
7 Function views
8     1. Add an import: from my_app import views
9     2. Add a URL to urlpatterns: path('', views.home, name='home')
10 Class-based views
11     1. Add an import: from other_app.views import Home
12     2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13 Including another URLconf
14     1. Import the include() function: from django.urls import include, path
15     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16 """
17 from django.contrib import admin
18 from django.urls import path, include
19
20 urlpatterns = [
21     path("admin/", admin.site.urls),
22     path("a1/", include("app.urls")),
23 ]
24
```

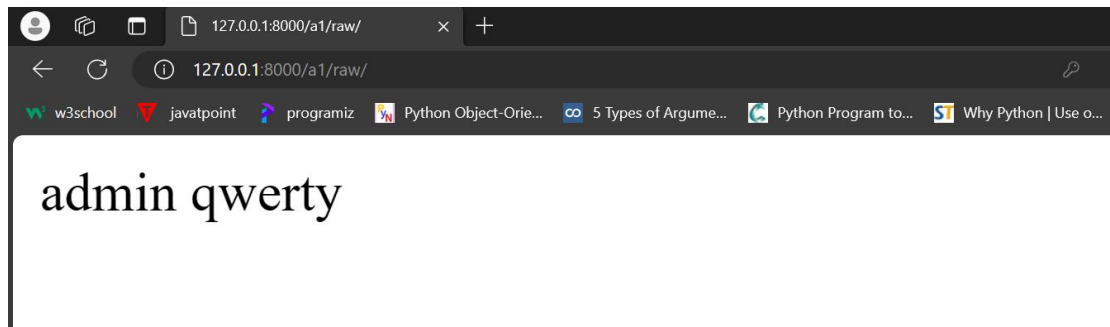
Step 6: Run your Django development server:

`python manage.py runserver`

Visit `http://127.0.0.1:8000/myform/` in your browser to access the form and test its functionality.

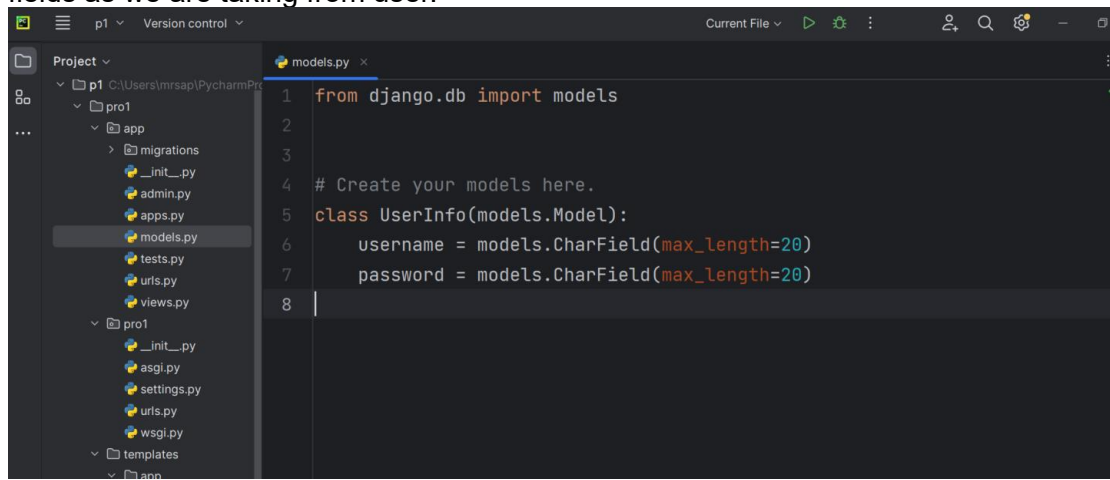


The screenshot shows a web browser window with the address bar displaying `127.0.0.1:8000/a1/raw/`. The page content includes a form with two input fields labeled "Username:" and "Password:", followed by a "Submit" button.



Step 7: Data Store into DataBase

- For that we should have model class defined in models.py Which has same fields as we are taking from user.

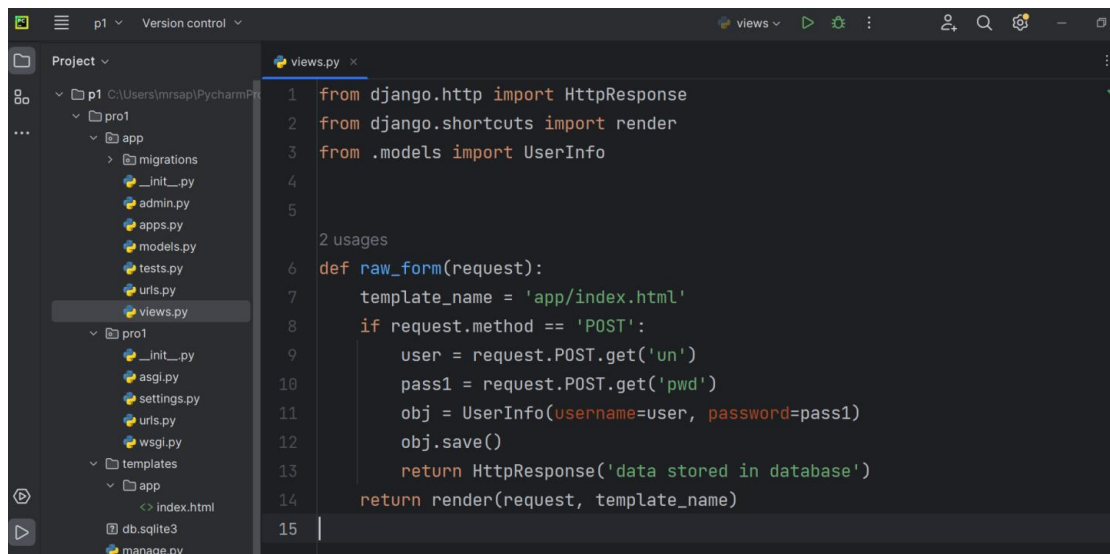


Command:

Py manage.py makemigrations

Py manage.py migrate

Step 8: Insert ORM query in views.py file



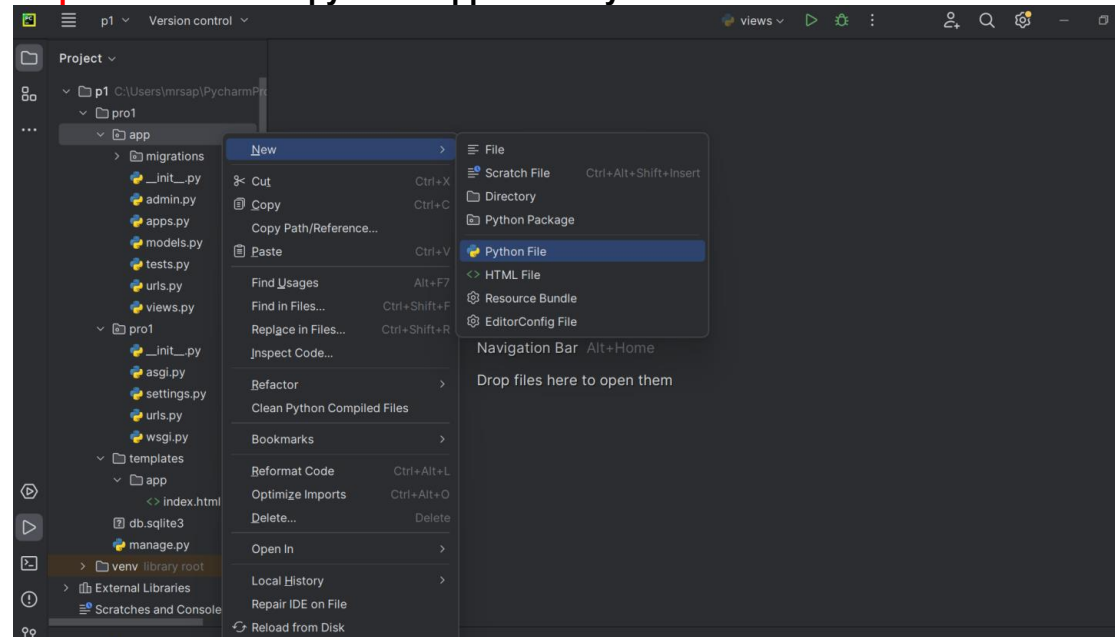
Django Form Class:

When one creates a Form class, the most important part is defining the fields of the form.

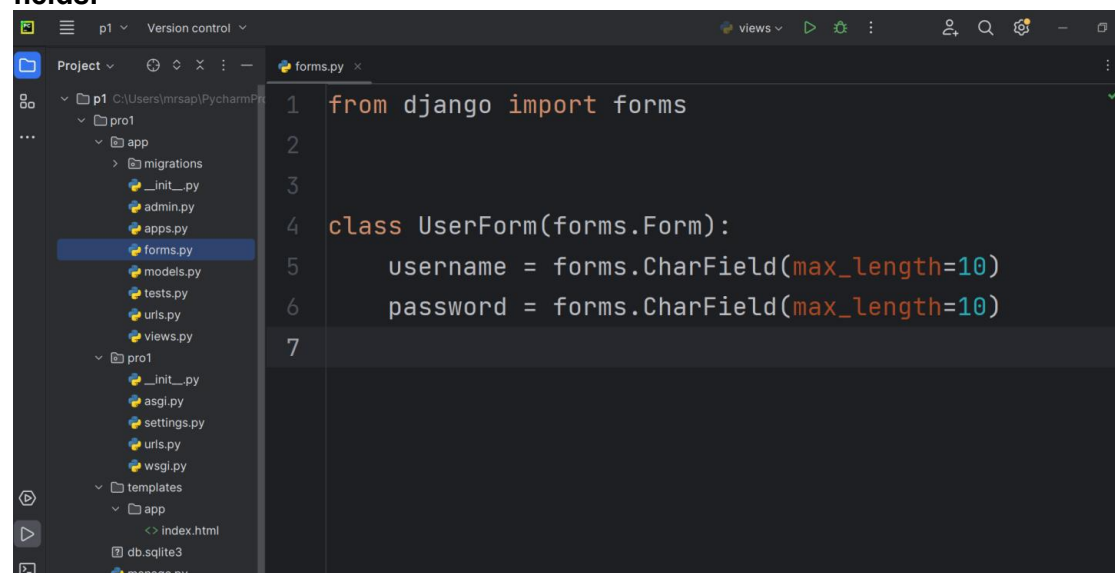
Forms are basically used for taking input from the user in some manner and using that information for logical operations on databases. For example, Registering a user by taking input as his name, email, password, etc.

Creating a form in Django is completely similar to creating a model, one needs to specify what fields would exist in the form and of what type. For example, to input, a registration form one might need First Name (CharField), Roll Number (IntegerField), and so on.

Step 1: Create forms.py file in app directory.



Step 2: After creating forms.py file in our application folder with our required fields.



- **UserForm** is a form class that inherits from **Form** class and imported from **forms.py**.
- Username, password are form fields defined as class attributes.(each field would be mapped as an input field in HTML)

- `forms.CharField`, `forms.EmailField`, and `forms.Textarea` are examples of field types provided by Django. They allow you to specify the type of data expected and provide validation rules.

The arguments that are common to most fields are listed below (these have sensible default values): **required**: If True, the field may not be left blank or given a None value. Fields are required by default, so you would set `required=False` to allow blank values in the form.

label: The label to use when rendering the field in HTML. If a label is not specified, Django will create one from the field name by capitalizing the first letter and replacing underscores with spaces (e.g. Renewal date).

label_suffix: By default, a colon is displayed after the label (e.g. Renewal date:). This argument allows you to specify a different suffix containing other character(s).

initial: The initial value for the field when the form is displayed.

widget: The display widget to use.

help_text (as seen in the example above): Additional text that can be displayed in forms to explain how to use the field.

error_messages: A list of error messages for the field. You can override these with your own messages if needed.

validators: A list of functions that will be called on the field when it is validated.

localize: Enables the localization of form data input (see link for more information).

disabled: The field is displayed but its value cannot be edited if this is True. The default is False.

Once you've defined your form class, you can use it in your views to render the form in a template, process submitted data, validate it, and perform any necessary actions.

Step 3: Define View

```

1  from django.http import HttpResponse
2  from django.shortcuts import render
3  from .models import UserInfo
4  from .forms import UserForm
5
6
7  def form_class_view(request):
8      template_name = 'app/index.html'
9      form = UserForm()
10     if request.method == 'POST':
11         form = UserForm(request.POST)
12         if form.is_valid():
13             user = form.cleaned_data.get('username')
14             pass1 = form.cleaned_data.get('password')
15             obj = UserInfo(username=user, password=pass1)
16             obj.save()
17             return HttpResponse('data stored in database')
18     context = {'form': form}
19     return render(request, template_name, context)
20

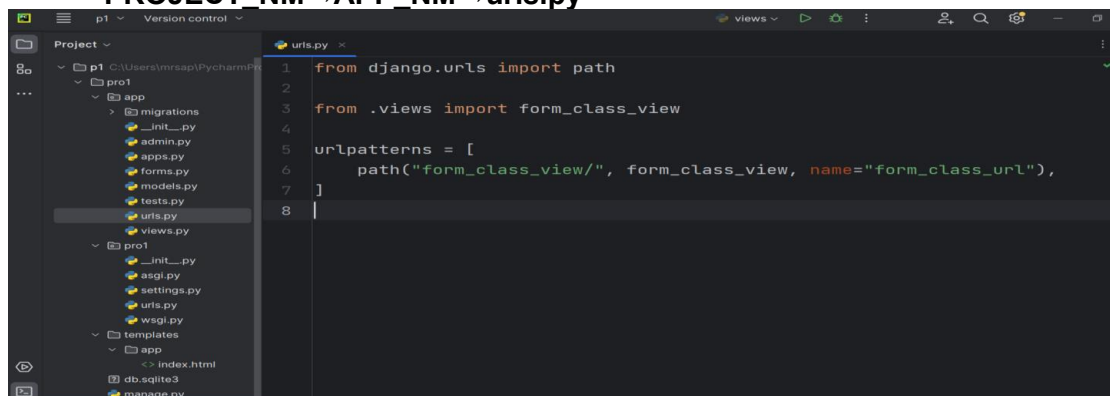
```

- On a **GET** request, an instance of **MyForm** is created and passed to the template for rendering.
- On a **POST** request, the form is instantiated with the data from the request. The **is_valid()** method is used to perform validation for each field of the form, it is defined in Django Form class. It returns True if data is valid and place all data into a **form.cleaned_data** attribute and perform further actions, such as saving to a database.
- Pass formclass object into the context.

```
{"form": formclass_nm}
```

Step 4: Define App_url

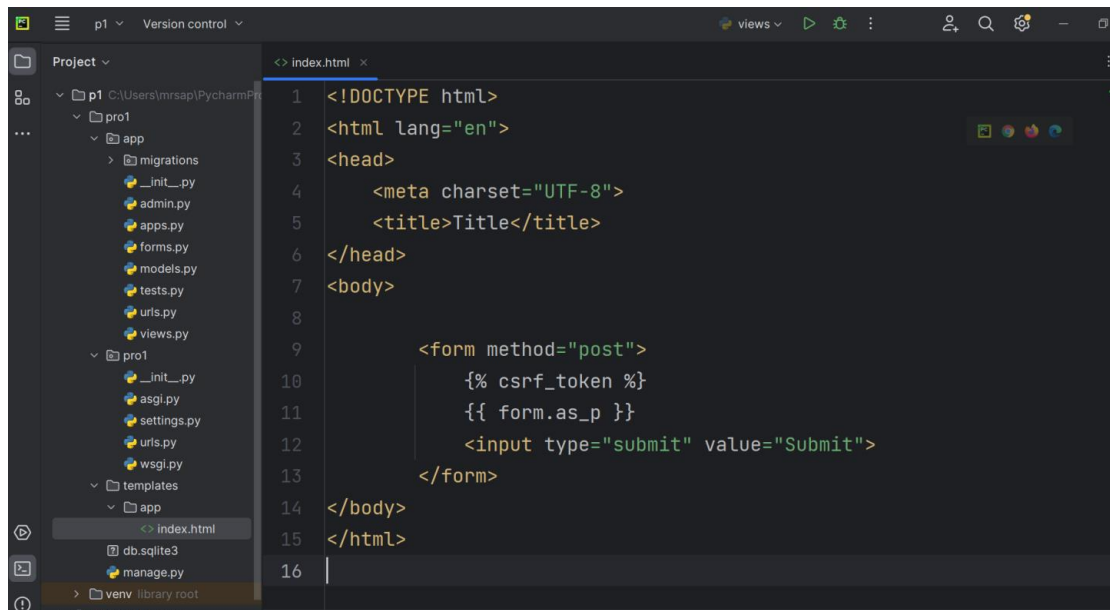
PROJECT_NM→APP_NM→urls.py



Step 5: Render Django Forms using Template

Django form fields have several built-in methods to ease the work of the developer but sometimes one needs to implement things manually for customizing User Interface(UI). A form comes with 3 in-built methods that can be used to render Django form fields.

- `{{ form.as_table }}` will render them as table cells wrapped in `<tr>` tags
- `{{ form.as_p }}` will render them wrapped in `<p>` tags
- `{{ form.as_ul }}` will render them wrapped in `<li>` tags

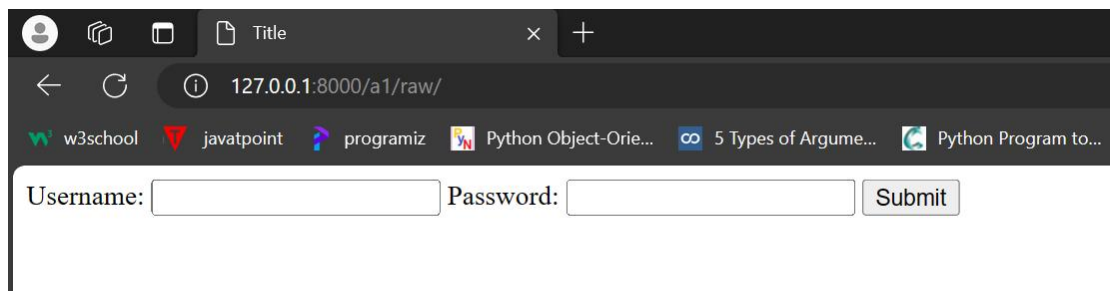


```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Title</title>
6 </head>
7 <body>
8
9     <form method="post">
10         {% csrf_token %}
11         {{ form.as_p }}
12         <input type="submit" value="Submit">
13     </form>
14 </body>
15 </html>
16
```

{% csrf_token %} is used for security reasons to prevent Cross-Site Request Forgery (CSRF) attacks. {{ form.as_p }} will render the form fields as paragraph elements (<p>).

Step 6: Run development server

Py manage.py runserver



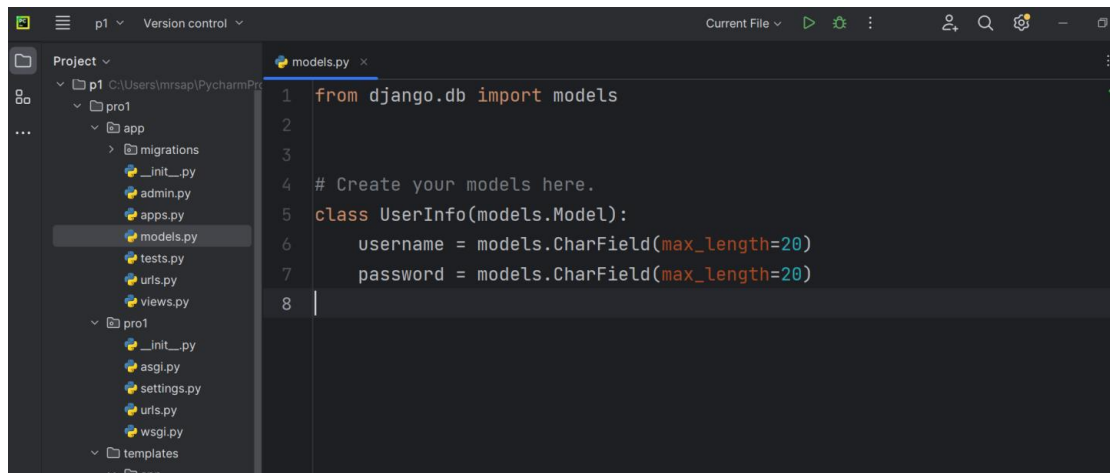
Django Model Form:

It is a class which is used to create an HTML form by using the Model. It is an efficient way to create a form without writing HTML code. Django automatically does it for us to reduce the application development time.

Step 1: Create a model

Create Model that contains fields name and other metadata. It can be used to create a table in database and dynamic HTML form.

// model.py



This file contains a class that inherits ModelForm and mention the model name for which HTML form is created.

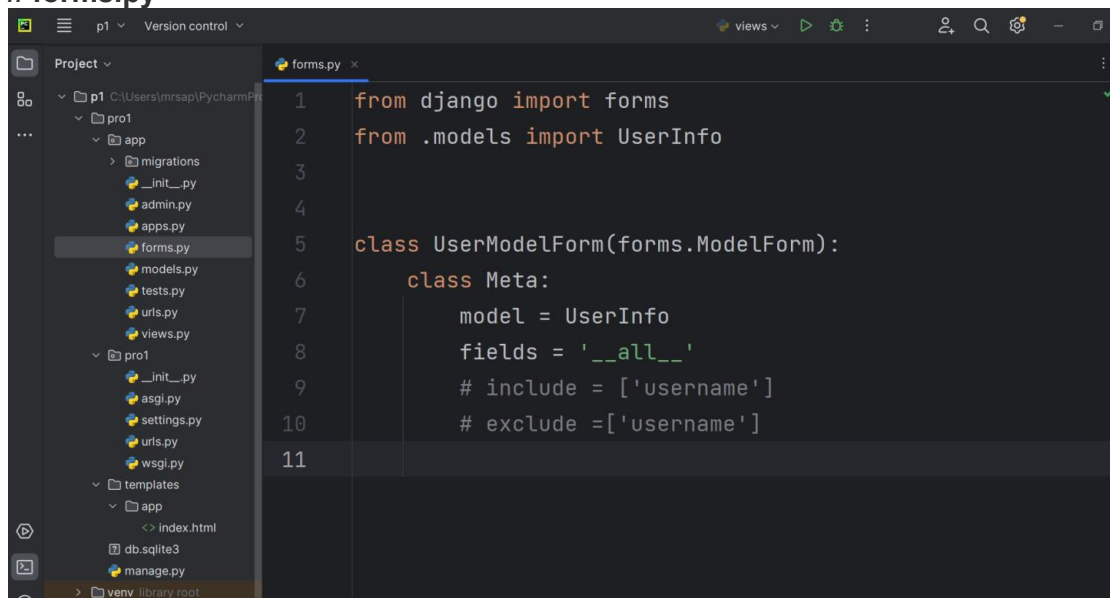
Step 2: Migrate

Py manage.py makemigrations

Py manage.py migrate

Step 3: Create Forms.py file

// forms.py



- **UserModelForm** is a form **class** is inherited from **ModelForm**.
- The **Meta class** is used to change the behavior of the **ModelForm**. Within it, specify the model your fields come from and the fields you want to use from that model.(to specify Model information and required fields.)
- field declarations if we are performing any custom validations.If we are not defining any custom validations then here we are not required to specify any field.

Case 1: All Fields

class Meta:

we have to specify Model class name and requied fields

model=Student

fields='__all__'

- **Case 2:** Instead of all fields if we want only selected fields, then we have to specify as follows

```
class Meta:
    model=Student
```

`fields=('field1','field2','field3')` #In the form only 3 fields will be considered.If Model class contains huge number of fields and we required to consider very less number of fields in the form then we should use this approach.

- **Case 3:** Instead of all fields if we want to exclude certain fields, then we have to specify as follows:

```
class Meta:
    model=Student
```

`exclude=['field1', 'field2']` #In the form all fields will be considered except field1 and field2.If the Model class contains huge number of fields and if we want to exclude very few fields then we have to use this approach

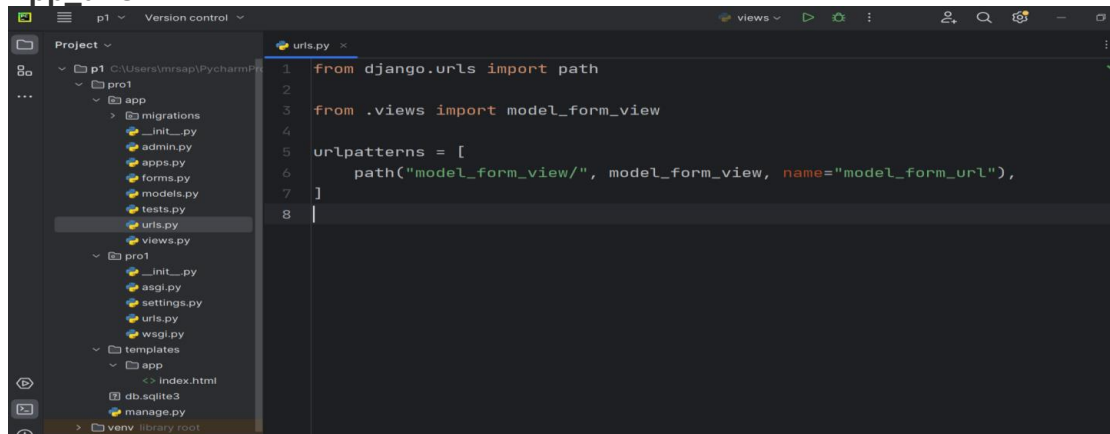
Step 4: Define views

//views.py

```
1 from django.http import HttpResponse
2 from django.shortcuts import render
3 from .models import UserInfo
4 from .forms import UserForm
5
6
7 def form_class_view(request):
8     template_name = 'app/index.html'
9     form = UserForm()
10    if request.method == 'POST':
11        form = UserForm(request.POST)
12        if form.is_valid():
13            user = form.cleaned_data.get('username')
14            pass1 = form.cleaned_data.get('password')
15            obj = UserInfo(username=user, password=pass1)
16            obj.save()
17            return HttpResponse('data stored in database')
18    context = {"form": form}
19    return render(request, template_name, context)
20
```

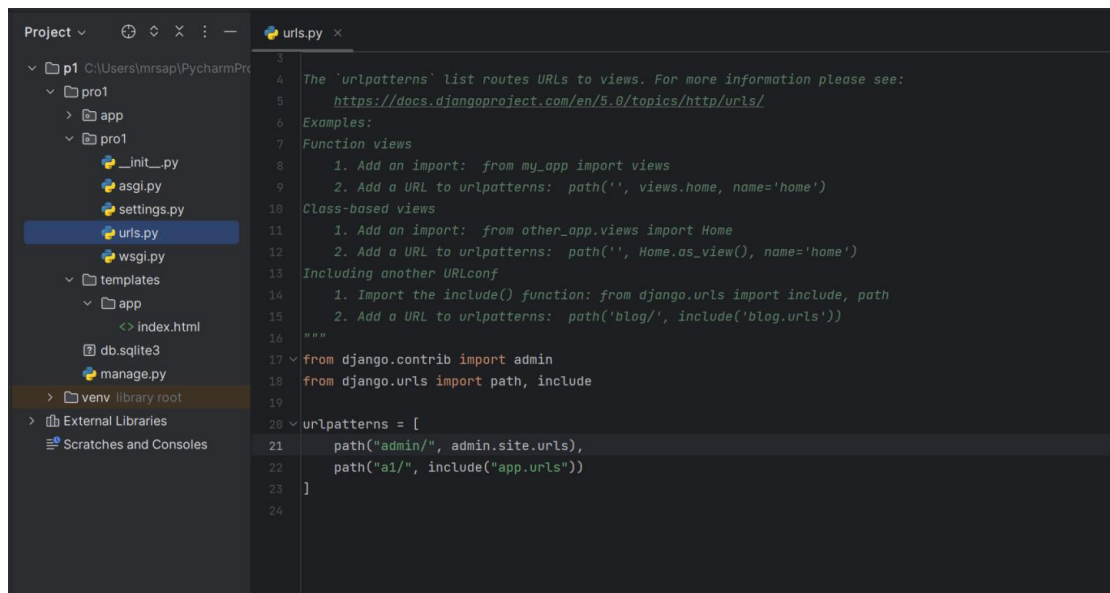
Step 5: Define urls

App_urls



```
1 from django.urls import path
2
3 from .views import model_form_view
4
5 urlpatterns = [
6     path("model_form_view/", model_form_view, name="model_form_url"),
7 ]
8
```

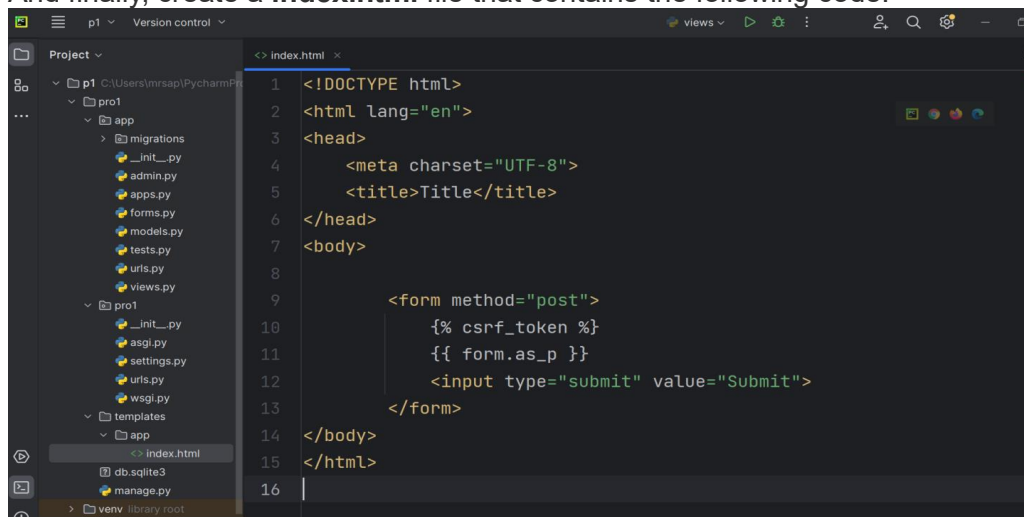
Project_urls



```
3
4 The 'urlpatterns' list routes URLs to views. For more information please see:
5     https://docs.djangoproject.com/en/5.0/topics/http/urls/
6 Examples:
7 Function views
8     1. Add an import: from my_app import views
9     2. Add a URL to urlpatterns: path('', views.home, name='home')
10 Class-based views
11     1. Add an import: from other_app.views import Home
12     2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13 Including another URLconf
14     1. Import the include() function: from django.urls import include, path
15     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16 """
17 from django.contrib import admin
18 from django.urls import path, include
19
20 urlpatterns = [
21     path("admin/", admin.site.urls),
22     path("a1/", include("app.urls")),
23 ]
24
```

Step 6: Define Template

And finally, create a **index.html** file that contains the following code.

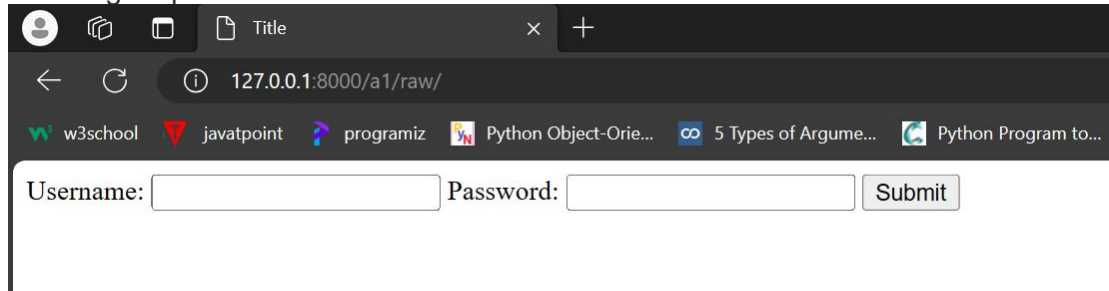


```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Title</title>
6 </head>
7 <body>
8
9     <form method="post">
10         {% csrf_token %}
11         {{ form.as_p }}
12         <input type="submit" value="Submit">
13     </form>
14 </body>
15 </html>
16
```

Step 7: Run Server

python manage.py runserver

After that access the template by **localhost:8000/index** URL, and it will display the following output to the browser.



Django Validation

Django Validation

Built-in methods are provided by Django for automatic validation of data of the form. Forms in Django are submitted only if it has a **CSRF** token. An easy approach is used by it for the form validation in Django. The **is_valid()** method is defined in the form class of Django and it is for the validation of the data of every field that the form has. If the data in the field is valid then true is returned by it and then it places data into an attribute **cleaned_data**

When you enter data, the browser and/or the web server will check to see that the data is in the correct format and within the constraints set by the application.

Validation done in the browser is called **client-side** validation(JavaScript, HTML, and CSS.), while validation done on the server is called **server-side** validation(PHP, Python, Node.js). client-side validation.

We can implement validation logic by using the following 2 ways.

1. Explicitly by the programmer by using clean methods
2. By using Django inbuilt validators

Note: All validations should be implemented in the **forms.py file**

Build_in Validators

Django's form (and model) fields support use of utility functions and classes known as validators. A validator is a callable object or function that takes a value and returns nothing if the value is valid or raises a **ValidationError** if not.

Built-in field validations in Django models are predefined in Django. They are default validations. Every field in Django has validations that are unique to it. The module **django.core.validators** contain variables that can be called. We can use the callable variables for the model and field. These callable variables are for internal use but can also be used for our field.

PROJNAME-->APPNAME-->forms.py

```
from django import forms
from django.core import validators
```

```
class NAMEForm(forms.Form):
    field1 = forms.DATATYPE(validators = [validators.ANYValidator,
                                          validators.ANYValidator2])
    field2 = forms.DATATYPE()
```


Built-in Field Validations

RegexValidator

Description

If the regex is not assigned as None, it allows regex overrides. This can be a pre-compiled regular expression.

If the message is not set as none, it will override the message. If the code is not assigned as none, it will enable overriding code.

If regex is not a regular expression string, a TypeError is raised.

The RegexValidator searches the provided value with re.search(). The RegexValidator, by default, gives a ValidationError with message and code. This is done when the match is not found. If the inverse_match is set to True, we can obtain an opposite result.

Syntax

Class

```
RegexValidator(regex=None, message=None)
```

EmailValidator

When not set to None, the message allows overriding of the message. Similarly, when code and allowlist are not set to None, it will enable overriding code and allowlist.

Class

```
EmailValidator(code=None, message=None, allowlist=None)
```

MaxValueValidator

A ValidationError is set with "max_value" when the limit_value is greater than the value.

class

```
MaxValueValidator(limit_value, message=None)
```

MinValueValidator

A ValidationError is set with "min_value" when the limit_value is less than the value.

class

```
MinValueValidator(limit_value, message=None)
```

MaxLengthValidator

A ValidationError is set with "max_length" when the

class

```
MaxLengthValidator(limit_value, message=None)
```

Custom Validators:

You can also create custom validators for more complex validation requirements. These are functions that take the field's value as input and raise a **ValidationError** if the validation fails.

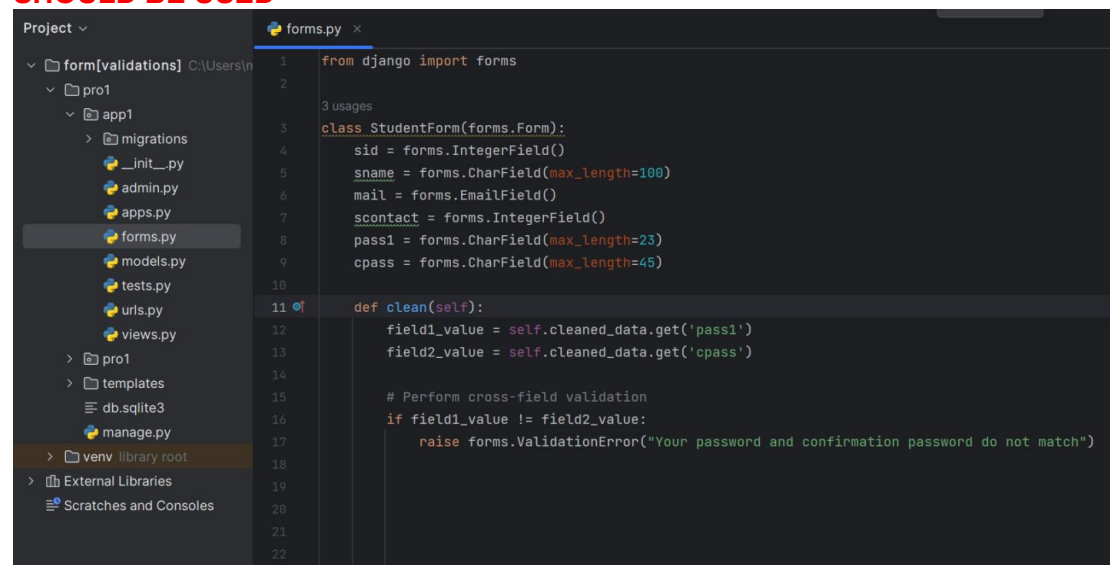
In Django, both `clean()` and `clean_<field_name>()` methods are used for form validation within Django forms or model forms.

Clean() Method:

1. This method is used for general validation across multiple fields in a form
- 2 This method returns the clean data, which is then inserted into the `cleaned_data` dictionary of the form.
- 3 When you override the `clean()` method in a Django form or model form, you're able to perform validation that involves multiple fields. This method is called after all the individual field validations (`clean_<field_name>()` methods) have been executed successfully.
- 4 You might use this method for cross-field validation, where the validity of one field depends on the value of another field.

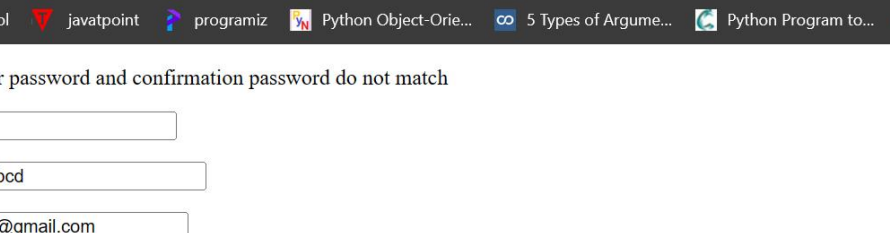
Syntax:

```
def clean(self):
all_data = super().clean()
entered_data1 = all_data['field1']
entered_data2 = all_data['field2'] #validation logic NO RETURN STATEMENT
SHOULD BE USED
```



```
Project ▾
  ▾ form[validations] C:\Users\h...
    ▾ pro1
      ▾ app1
        ▾ migrations
          __init__.py
        admin.py
        apps.py
        forms.py
        models.py
        tests.py
        urls.py
        views.py
      ▾ pro1
      ▾ templates
      db.sqlite3
      manage.py
    ▾ venv library root
    ▾ External Libraries
    Scratches and Consoles

forms.py x
1 from django import forms
2
3 usages
4 class StudentForm(forms.Form):
5     sid = forms.IntegerField()
6     sname = forms.CharField(max_length=100)
7     mail = forms.EmailField()
8     scontact = forms.IntegerField()
9     pass1 = forms.CharField(max_length=23)
10    cpass = forms.CharField(max_length=45)
11
12    def clean(self):
13        field1_value = self.cleaned_data.get('pass1')
14        field2_value = self.cleaned_data.get('cpass')
15
16        # Perform cross-field validation
17        if field1_value != field2_value:
18            raise forms.ValidationError("Your password and confirmation password do not match")
19
20
21
22
```



The screenshot shows a web browser window with a dark theme. The address bar displays the URL "127.0.0.1:8000/a1/v1/". The browser's tab bar shows several open tabs, including "w3school", "javatpoint", "programiz", "Python Object-Orie...", "5 Types of Argume...", "Python Program to...", and "Why Python". The main content area of the browser displays a registration form with the following elements:

- A red error message: "• Your password and confirmation password do not match".
- A "Sid:" label followed by a text input field containing the value "25".
- A "Sname:" label followed by a text input field containing the value "Abcd".
- A "Mail:" label followed by a text input field containing the value "aaa@gmail.com".
- A "Scontact:" label followed by a text input field containing the value "9156787890".
- A "Pass1:" label followed by a text input field containing the value "abc".
- A "Cpass:" label followed by a text input field containing the value "abcd".
- A "Submit" button at the bottom of the form.

Clean_<fieldname>() Method:

1. This method is used for field-specific validation.
2. Django allows you to define methods named `clean_<field_name>()` for individual fields. These methods perform validation specific to that field.
3. These methods are automatically called by Django during form validation.
4. In the FormClass for any field if we define clean method then at the time of submit the form, Django will call this method automatically to perform validations. If the clean method won't raise any error then only form will be submitted.
5. This method has no parameters.

Syntax:

```
def clean_field1(self):
```

```
entered_data= self.cleaned_data['field1']
```

return value

#validation logic with entered_data MANDATORILY RETURN VALUE

A screenshot of a code editor interface. On the left, a project explorer shows a directory structure with files like migrations, __init__.py, admin.py, apps.py, forms.py (highlighted), models.py, tests.py, urls.py, and views.py. The main editor area shows the content of forms.py, which defines a Django form class StudentForm. The class has several fields: sid (IntegerField), sname (CharField with max_length=100), mail (EmailField), scontact (IntegerField), pass1 (CharField with max_length=23), and cpass (CharField with max_length=45). There are two validation methods: clean_sid, which raises a ValidationError if sid is less than or equal to 0, and clean_sname, which raises a ValidationError if the first letter of sname is not uppercase. The code is written in Python with standard Django form syntax.

← 127.0.0.1:8000/a1/v1/

• ID should be greater than 0

Sid:

• First letter should be UPPERCASE

Sname:

Mail:

Scontact:

Pass1:

Cpass:

ModelForm Validation

1. Similarly we can apply validations on modelform by defining clean, clean_<fieldname> or using validators.
2. We also can apply validators on model class fields , see below.

Build in Validators:

The screenshot shows the PyCharm IDE with a project named 'form[validations]'. The file explorer on the left shows the project structure, including 'models.py'. The main editor window displays the following Python code in 'models.py':

```

1 from django.db import models
2 from django.core import validators
3
4 # Create your models here.
5
6 class Student(models.Model):
7     sid = models.IntegerField(validators=[validators.MinValueValidator(20)])
8     sname = models.CharField(max_length=100, validators=[validators.MaxLengthValidator(10)])
9     mail = models.EmailField()
10
11

```

← 127.0.0.1:8000/a2/v1/

• Ensure this value is greater than or equal to 20.

Sid:

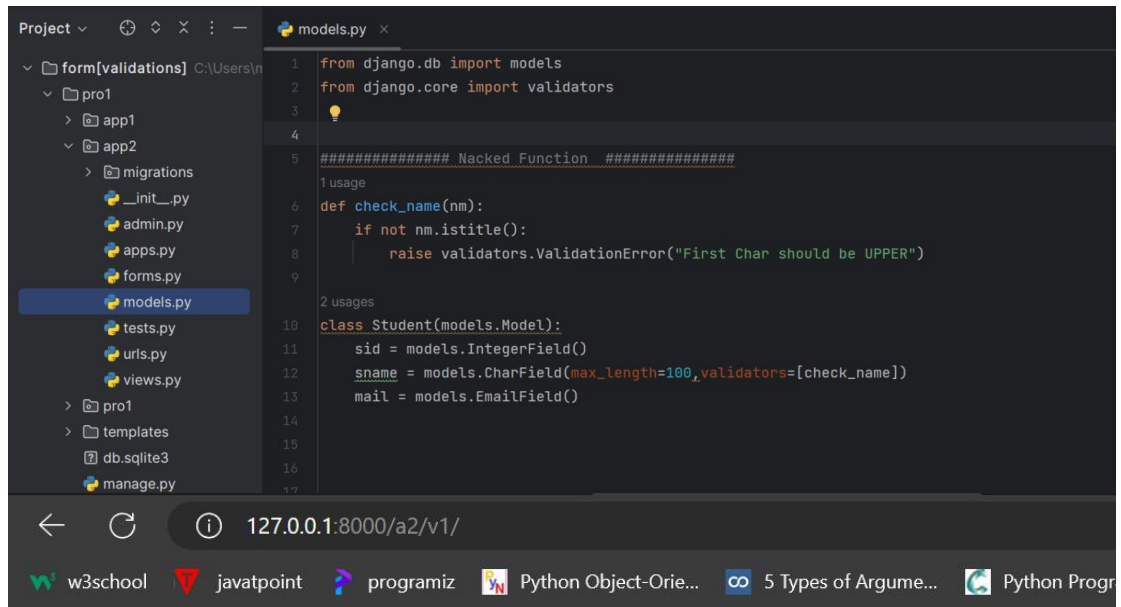
• Ensure this value has at most 10 characters (it has 11).

Sname:

• Enter a valid email address.

Mail:

Custom Validators



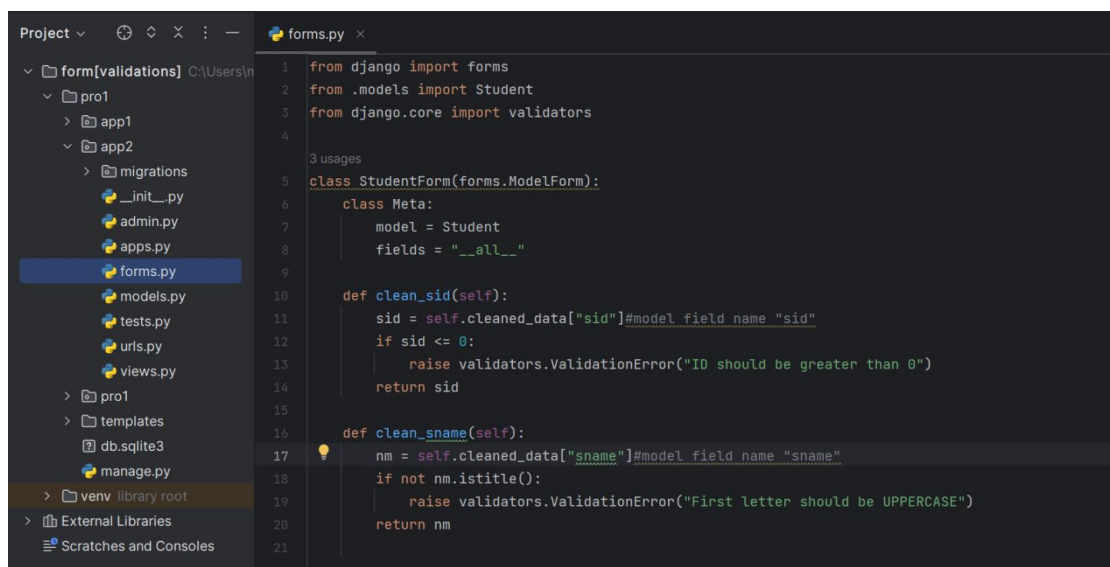
```
1 from django.db import models
2 from django.core import validators
3
4
5 ##### Nacked Function #####
6
7 1 usage
8 def check_name(nm):
9     if not nm.istitle():
10         raise validators.ValidationError("First Char should be UPPER")
11
12 2 usages
13 class Student(models.Model):
14     sid = models.IntegerField()
15     sname = models.CharField(max_length=100, validators=[check_name])
16     mail = models.EmailField()
```

Sid:

- First Char should be UPPER

Sname:

Mail:



```
1 from django import forms
2 from .models import Student
3 from django.core import validators
4
5 3 usages
6 class StudentForm(forms.ModelForm):
7     class Meta:
8         model = Student
9         fields = "__all__"
10
11 def clean_sid(self):
12     sid = self.cleaned_data["sid"]#model field name "sid"
13     if sid <= 0:
14         raise validators.ValidationError("ID should be greater than 0")
15     return sid
16
17 def clean_sname(self):
18     nm = self.cleaned_data["sname"]#model field name "sname"
19     if not nm.istitle():
20         raise validators.ValidationError("First letter should be UPPERCASE")
21     return nm
```

← ↻ ⓘ 127.0.0.1:8000/a2/v1/

w3school javatpoint programiz Python Object-Orie... 5 Types of Argume... Python Program to... Why Python | Use

- ID should be greater than 0

Sid:

- First letter should be UPPERCASE

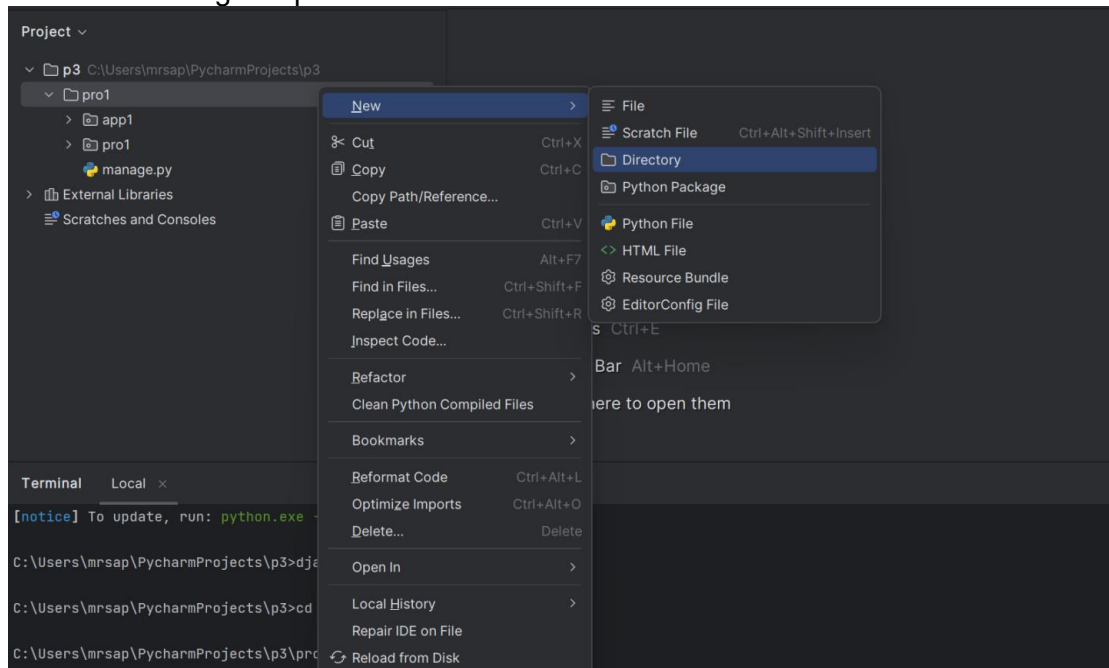
Sname:

Mail:

Static-Files

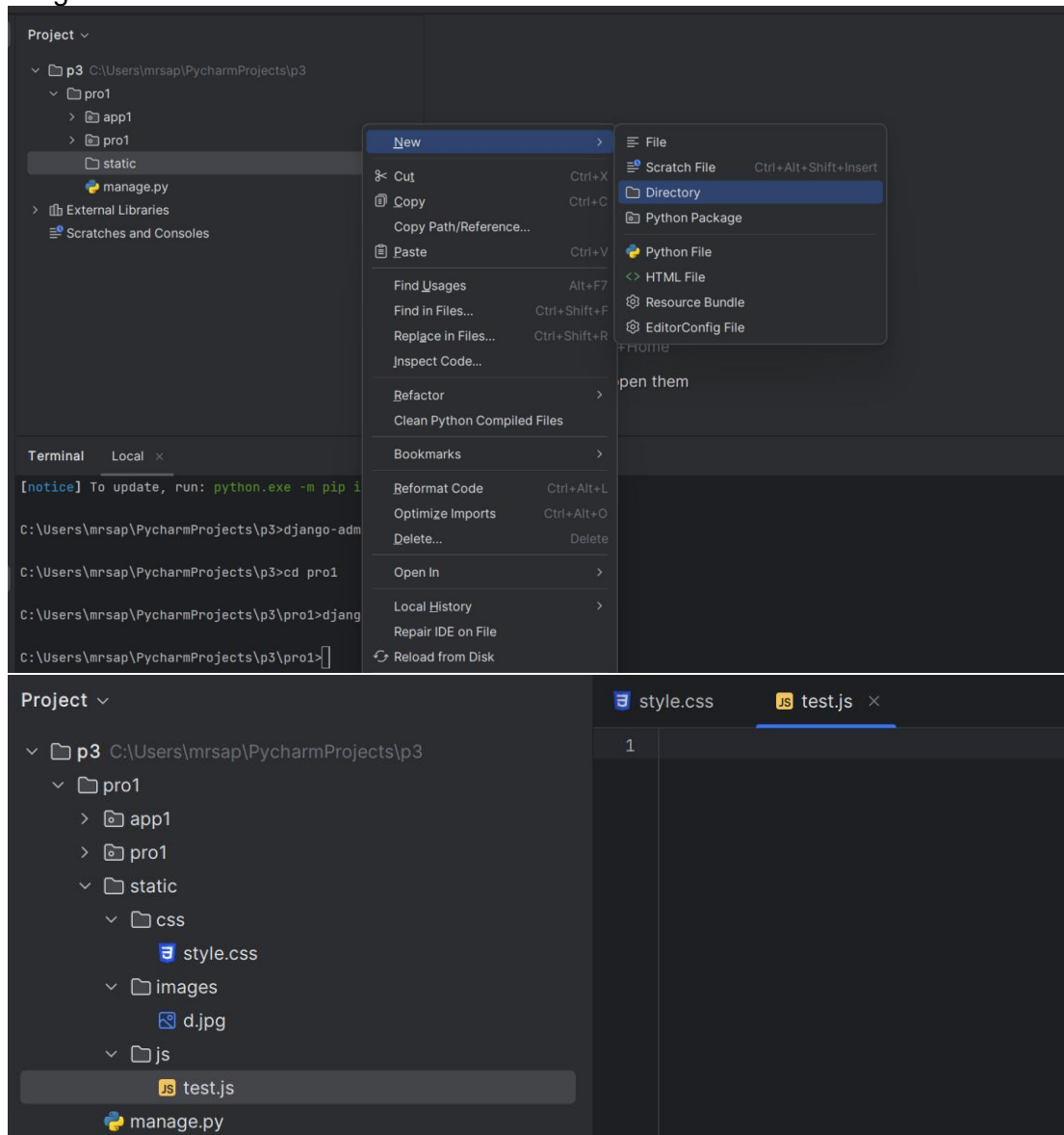
- Static files are files that don't change when your application is running.
- Static Files such as Images, CSS, or JS files are often loaded via a different app in production websites to avoid loading multiple stuff from the same server. This article revolves around, how you can set up the static app in Django and server Static Files from the same.

Step 1:- Create a folder named with 'static' inside main project folder. It is exactly same as creating 'templates' folder.



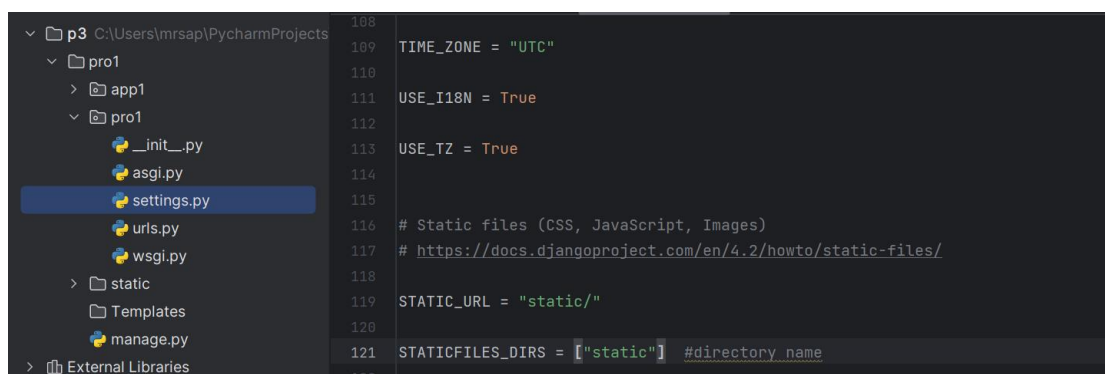
Step 2:- In that 'static' folder create 'image', 'CSS', 'JS ' folder to place image files,css file,etc.

Right click on "image" folder to open the file in export form and insert img-file in image folder.



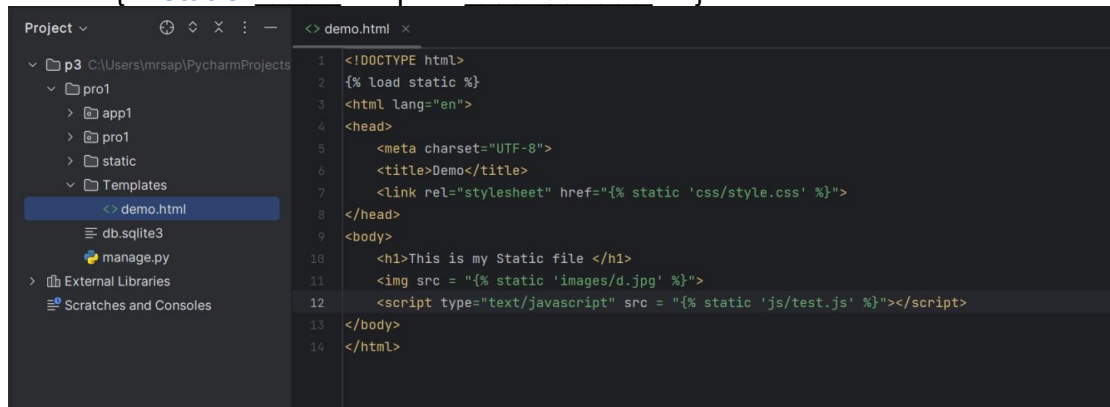
Step 3:- Add static directory path to settings.py file, so that Django can aware of our images.

PROJNAME->PROJNAME->settings.py->STATICFILES_DIRS = ['static']

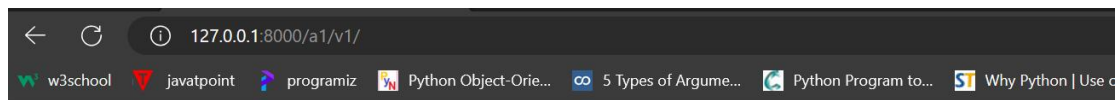


Step 4:- Load static files in the templates by using the below expression.

`{% load static %}`
`"{% static 'file path' %}"`



```
1 <!DOCTYPE html>
2 {% load static %}
3 <html lang="en">
4 <head>
5     <meta charset="UTF-8">
6     <title>Demo</title>
7     <link rel="stylesheet" href="{% static 'css/style.css' %}">
8 </head>
9 <body>
10     <h1>This is my Static file </h1>
11     
12     <script type="text/javascript" src="{% static 'js/test.js' %}"></script>
13 </body>
14 </html>
```



This is my Static file

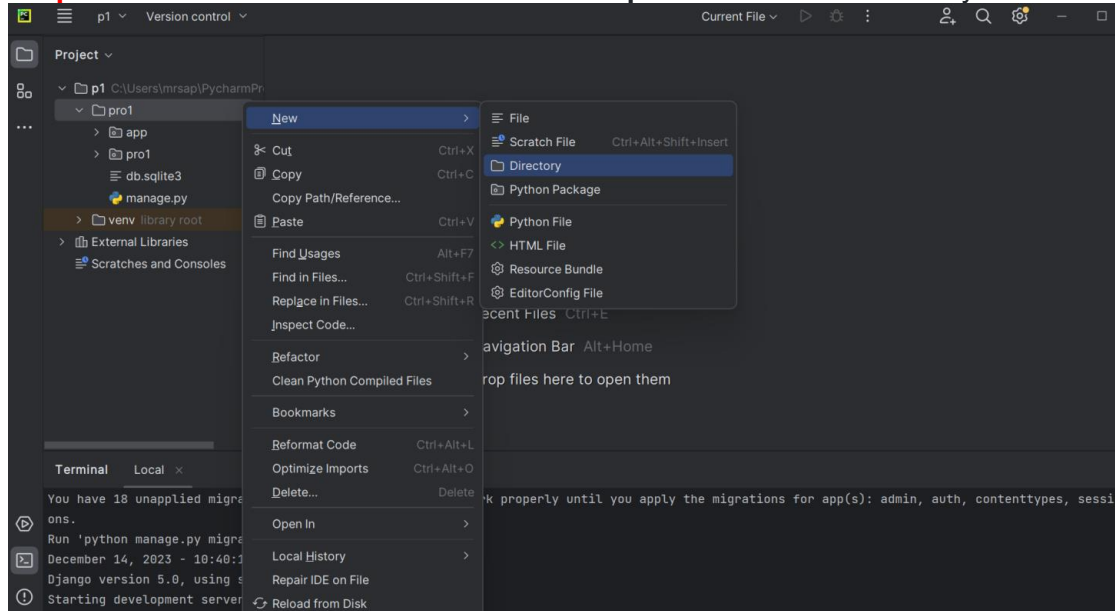


Templates in Django

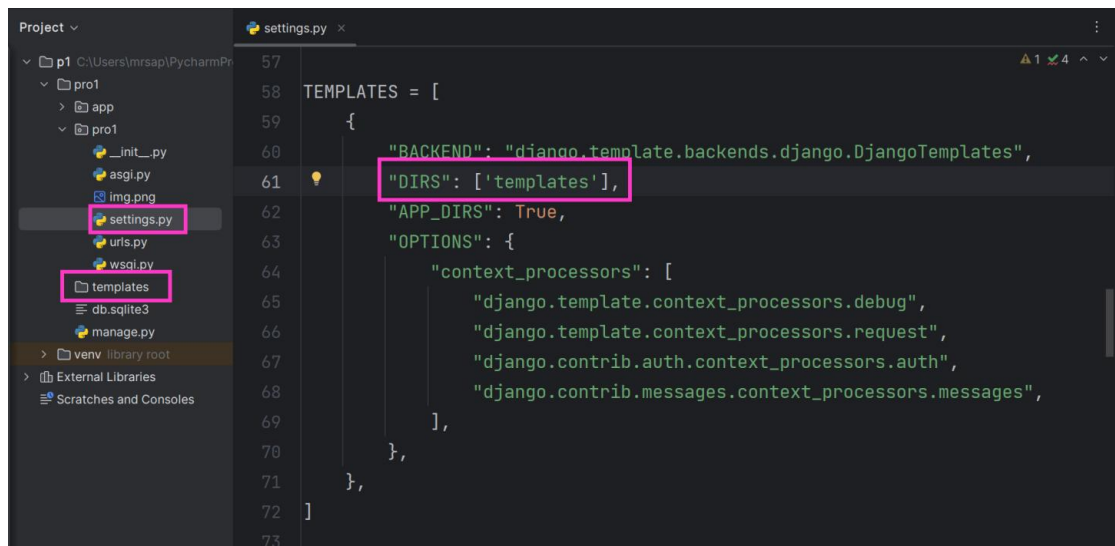
Templates in Django

We learned how to return a `HttpResponse` with a `h1` tag from a view. To return a full HTML page, you'll need to use a templates.

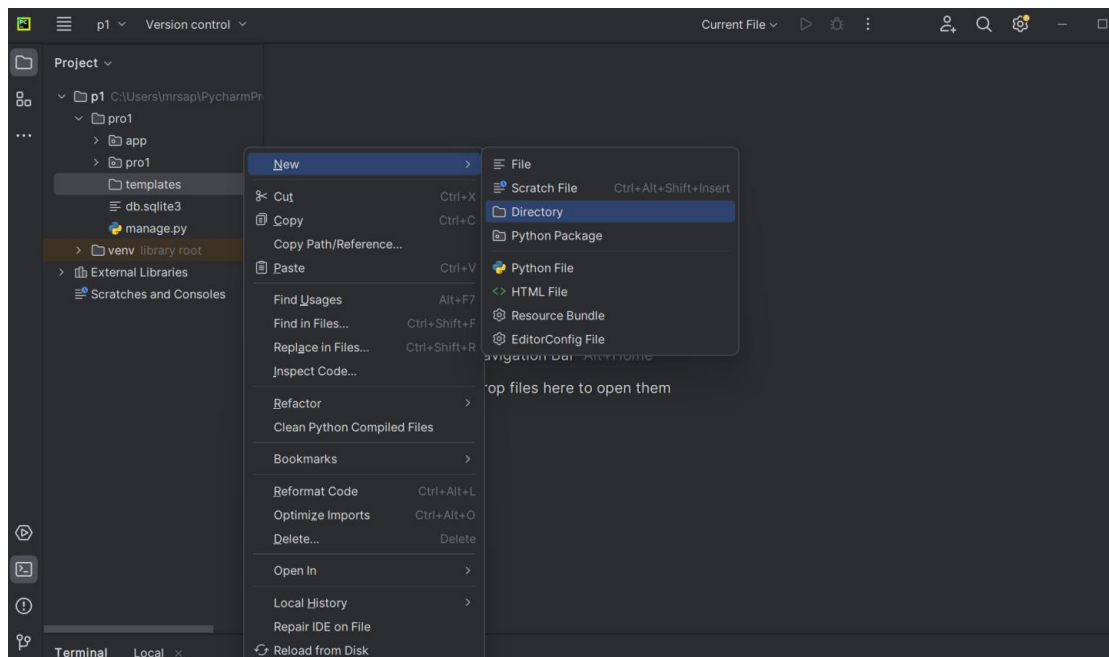
Step 1:- we need to create a folder named **templates** inside root directory.



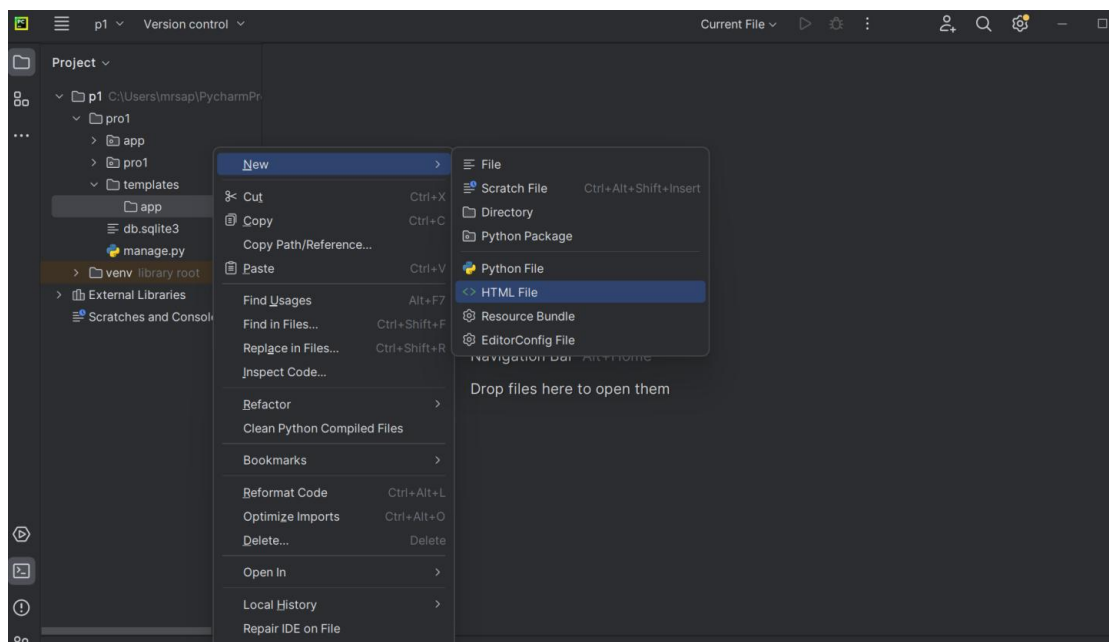
Step 2:- Register our templates folder in setting.py inside the `TEMPLATES` list .
So that Django will search for html files inside templates directory



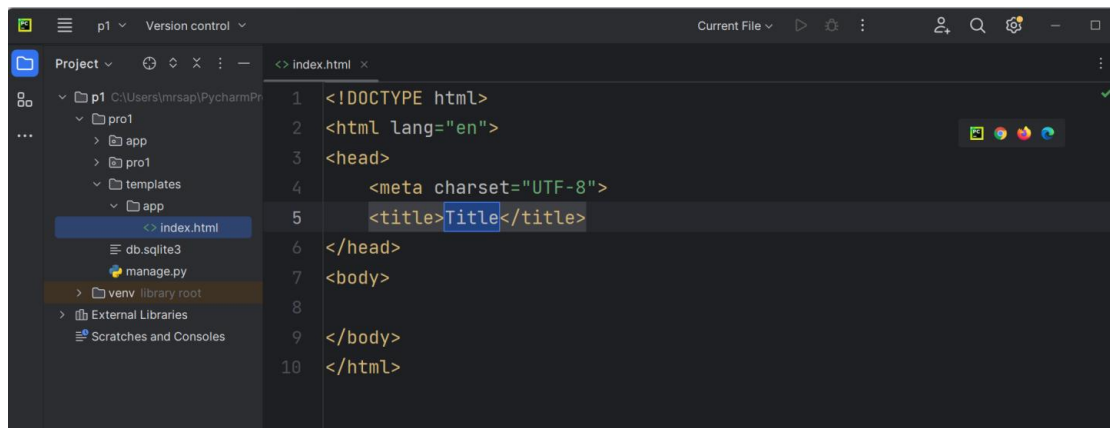
Step 3 :- Click on templates directory create new folder named it same like app so that we can keep html files separate required for diff. apps for maintenance purpose.



Step 4:- Right click on app create new html file index.html(you can name it anything)

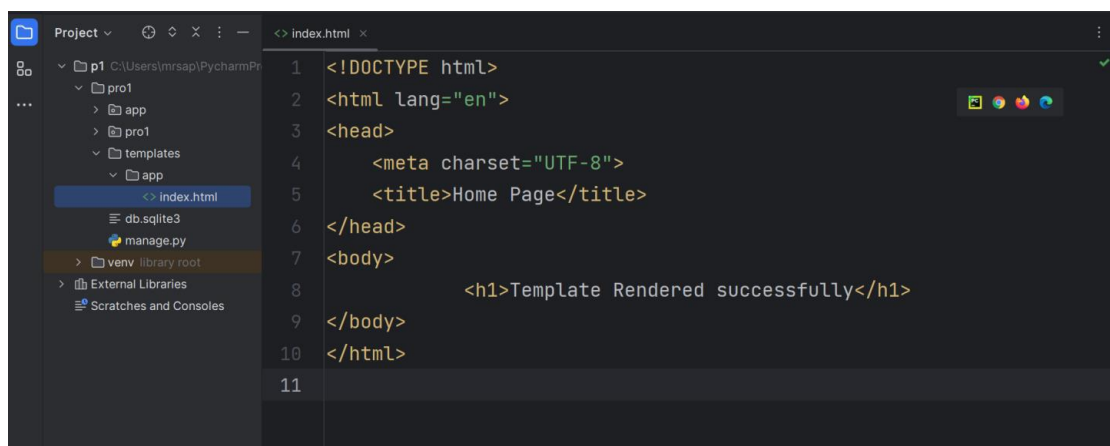


After creation of html file you will get default code (boilerplate) modify that code.



This screenshot shows the PyCharm IDE with a project named 'p1'. The file explorer on the left shows the project structure, including 'app', 'templates', and 'app' subdirectories. The main editor displays the 'index.html' file with the following content:

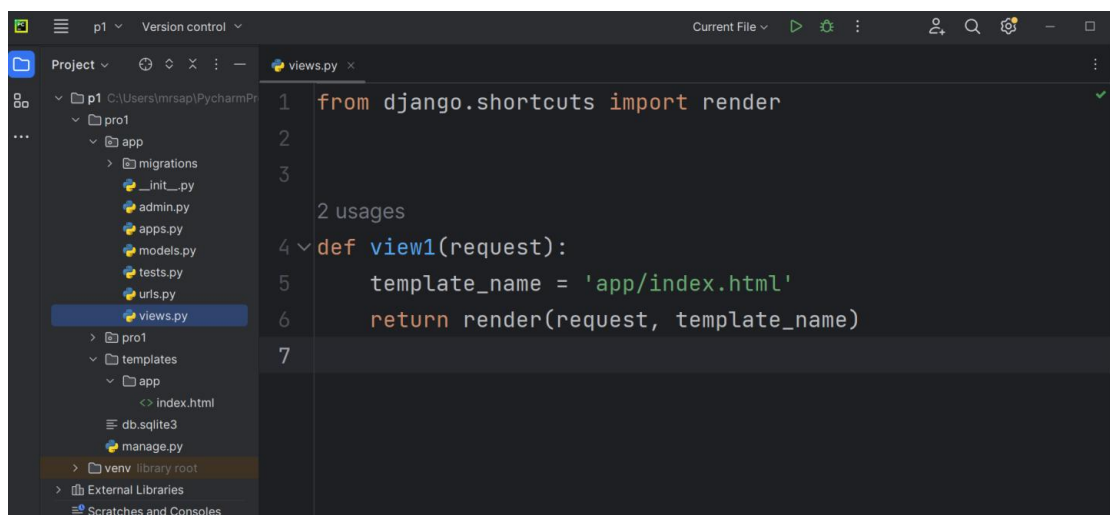
```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Title</title>
6 </head>
7 <body>
8
9 </body>
10 </html>
```



This screenshot shows the PyCharm IDE with the same project structure. The 'index.html' file has been updated with the following content:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Home Page</title>
6 </head>
7 <body>
8     <h1>Template Rendered successfully</h1>
9 </body>
10 </html>
11
```

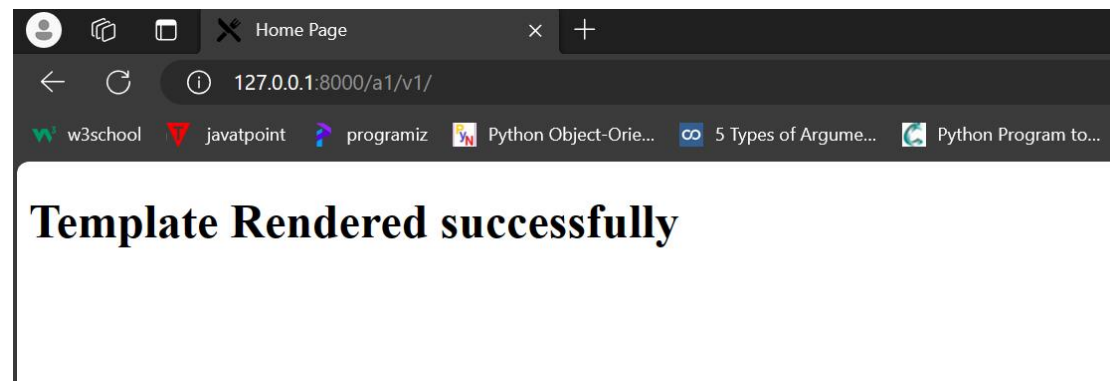
Now instead of HttpResponse Objcet we are going to use render function which we again internally returning an HttpResponse.
There are two required arguments for render function request and template_name.



This screenshot shows the PyCharm IDE with the same project structure. The 'views.py' file has been updated with the following content:

```
1 from django.shortcuts import render
2
3 2 usages
4 def view1(request):
5     template_name = 'app/index.html'
6     return render(request, template_name)
7
```

Urls will remain same now runserver and template will get render on browser.



DTL(Django template language)

- DTL is a powerful and flexible language that allows you to create dynamic web pages using HTML templates.
- At runtime, the template engine replaces variables in a template file with actual values, and transforms the template into an HTML file sent to the client.

The Basic Structure of Django Template Language(DTL)

The DTL syntax is very similar to Python. It consists of :

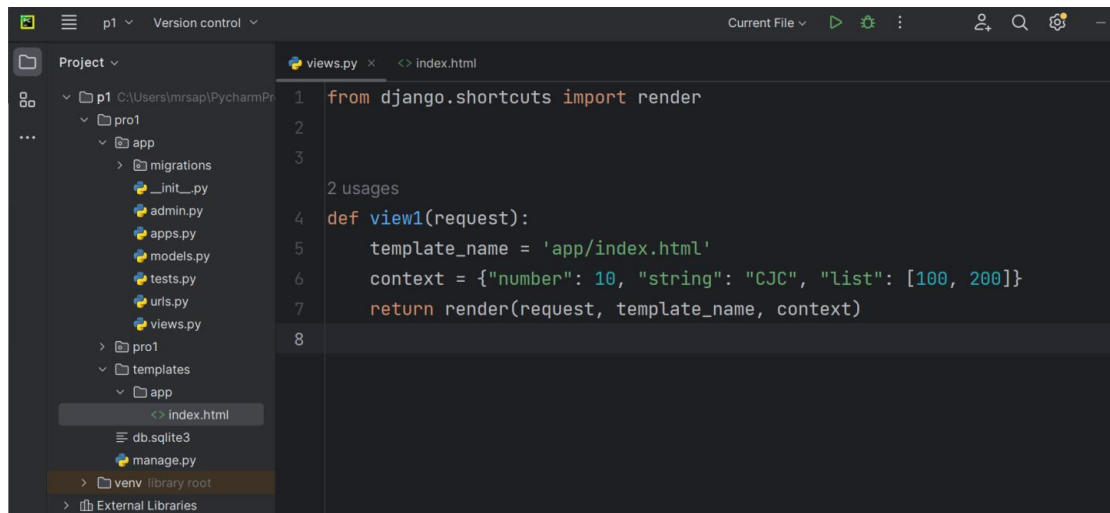
- Template tags --> {% tag %}
- Template variables --> {{ var }}
- Template Filters --> {{ var|lower }}
- Template Comments --> {%# #}

The template tags are a way of telling Django that here comes something else than plain HTML. The template tags allows us to do some programming on the server before sending HTML to the client.

Before using DTL lets learn about context dictionary which is third argument of our render function.

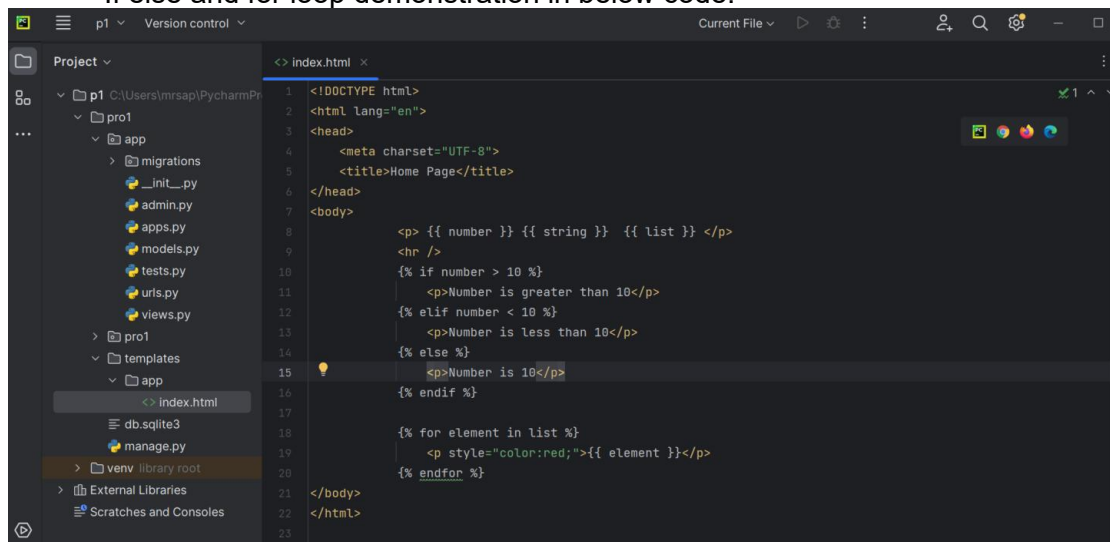
Context

- A **Context** is a dictionary with variable names as the **key** and their values as the **value**.
- Variable(Key) names consist of any combination of alphanumeric characters and the underscore (" _")
- Do not start with an underscore, and number.
- You cannot have spaces or punctuation characters "." in variable names.



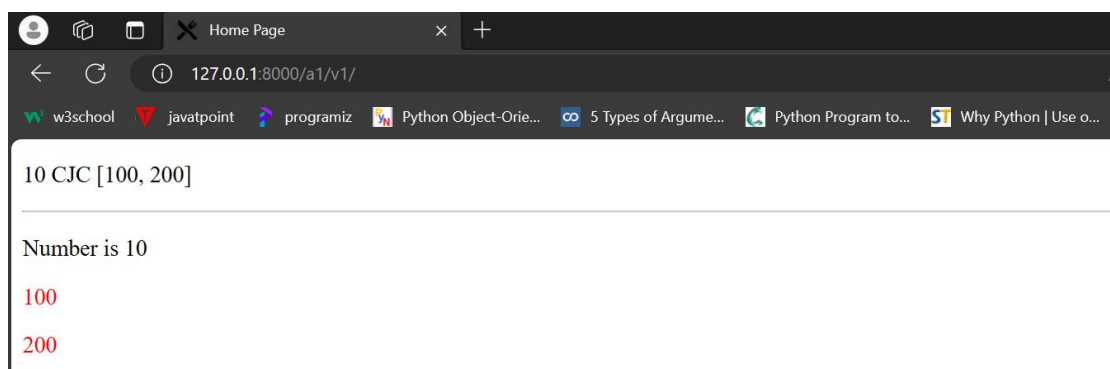
```
1 from django.shortcuts import render
2
3
4 2 usages
5 def view1(request):
6     template_name = 'app/index.html'
7     context = {"number": 10, "string": "CJC", "list": [100, 200]}
8     return render(request, template_name, context)
```

- If else and for loop demonstration in below code.



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Home Page</title>
6 </head>
7 <body>
8     <p> {{ number }} {{ string }} {{ list }} </p>
9     <hr />
10    {% if number > 10 %}
11        <p>Number is greater than 10</p>
12    {% elif number < 10 %}
13        <p>Number is less than 10</p>
14    {% else %}
15        <p>Number is 10</p>
16    {% endif %}
17
18    {% for element in list %}
19        <p style="color:red">{{ element }}</p>
20    {% endfor %}
21 </body>
22 </html>
```

Keep in mind that → this is invalid `{{ }}`, `{ % %}` there should be space between curly braces and % symbol.

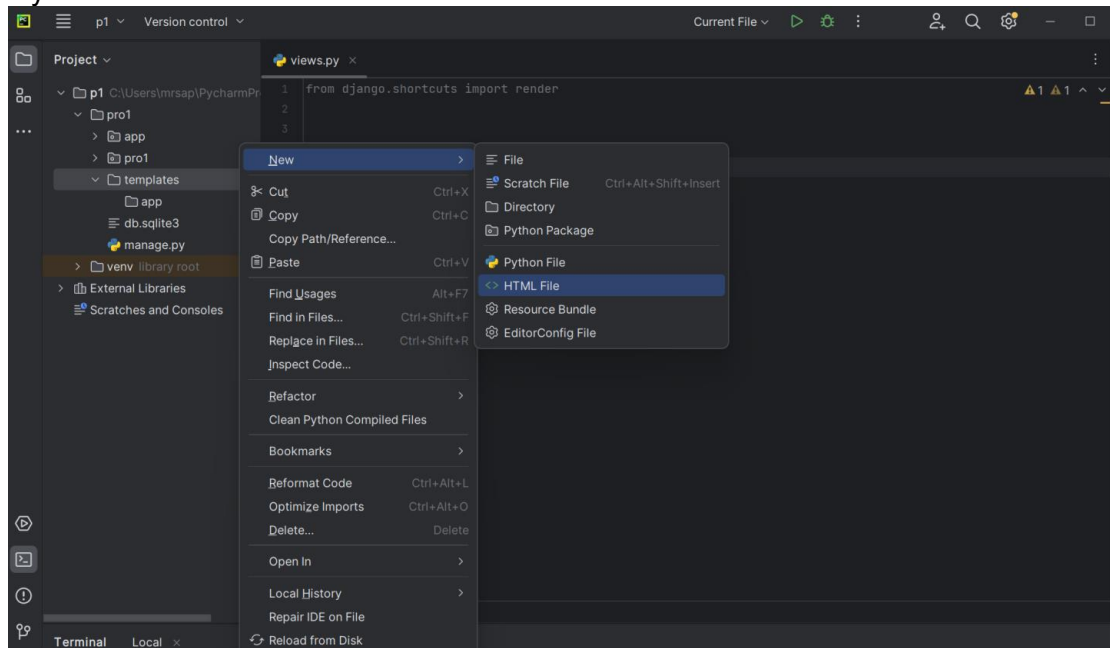


Template inheritance

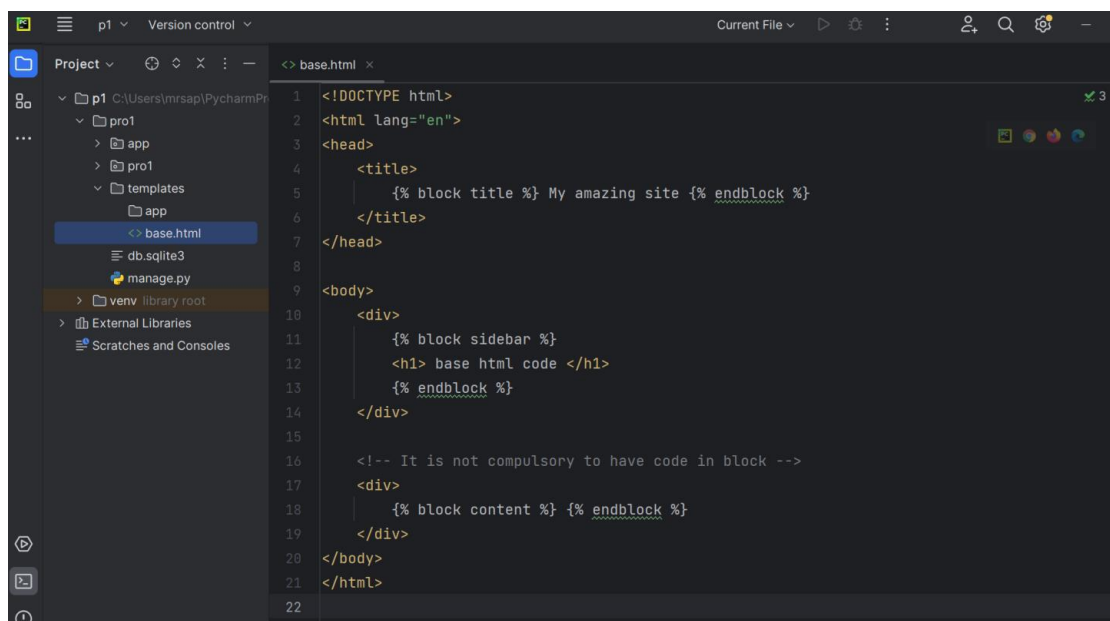
Template inheritance allows you to build a base “skeleton” template that contains all the common elements of your site and defines **blocks** that child templates can override.

Let’s look at template inheritance by starting with an example:

Step 1:- Create new html file inside templates directory by the name base / layout.html.



base.html



It's the job of "child" templates to fill the empty blocks with content. In this example, the `block` tag defines three blocks that child templates can fill in. All the `block` tag does is to tell the template engine that a child template may override those portions of the template.

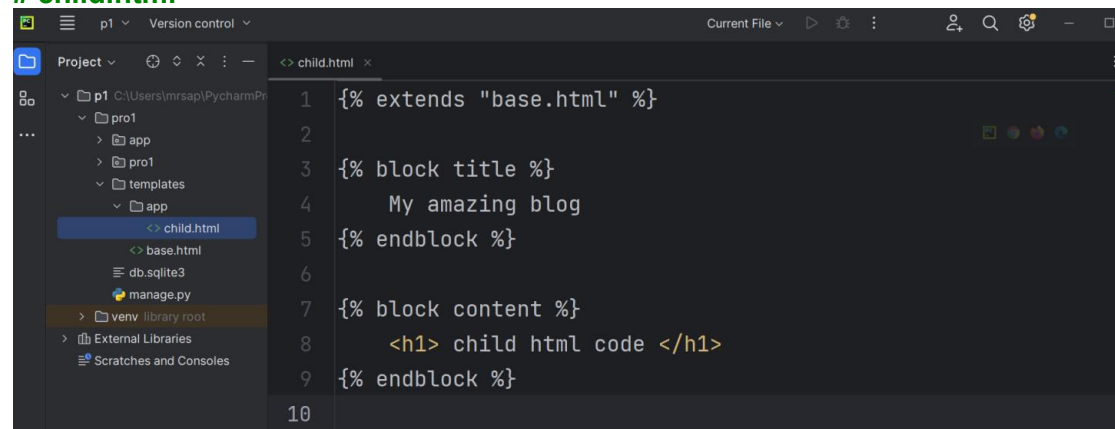
Note :

- We can override only those blocks which are defined in parent template.
- It is not necessary to override all blocks of parent
- If you use `{% extends "parent.html" %}` in a template, it must be the first template tag in that template. Template inheritance won't work, otherwise.

Step 2:- Create child template inside app directory in templates

A child template might look like this:

child.html

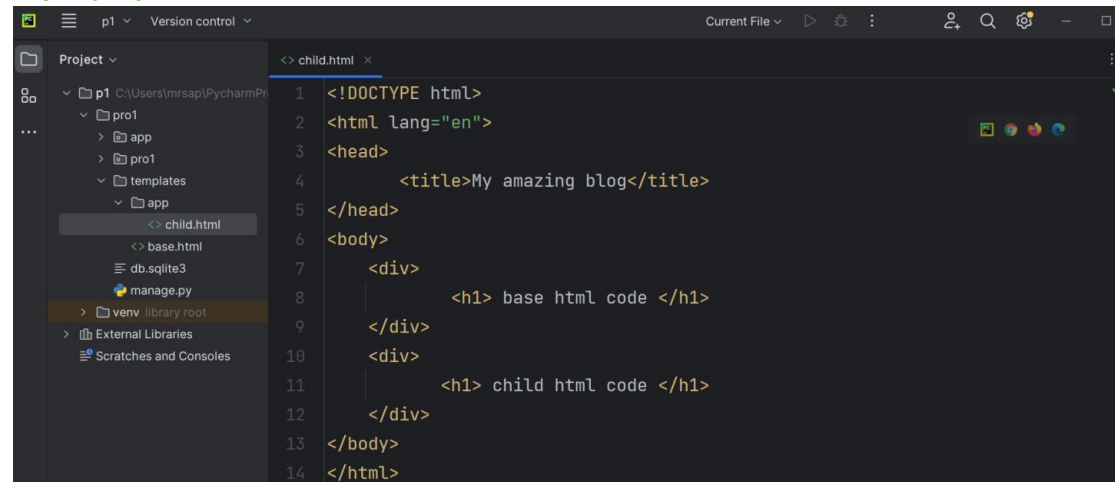


```
1 {% extends "base.html" %}
2
3 {% block title %}
4     My amazing blog
5 {% endblock %}
6
7 {% block content %}
8     <h1> child html code </h1>
9 {% endblock %}
10
```

The `extends` tag is the key here. It tells the template engine that this template "extends" another template. When the template system evaluates this template, first it locates the parent – in this case, "base.html".

At that point, the template engine will notice the three `block` tags in base.html and replace those blocks with the contents of the child template. Depending on the value the output might look like:

child.html



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <title>My amazing blog</title>
5 </head>
6 <body>
7     <div>
8         <h1> base html code </h1>
9     </div>
10    <div>
11        <h1> child html code </h1>
12    </div>
13 </body>
14 </html>
```


Note that since the child template didn't define the `first div` block, the value from the parent template is used instead. Content within a `{% block %}` tag in a parent template is always used as a fallback

Relationship In Django:

There are 3 types of relationship :

1. One-to-One Relationship
2. One-to-Many Relationship
3. Many-to-Many Relationship

One-to-One Relationship

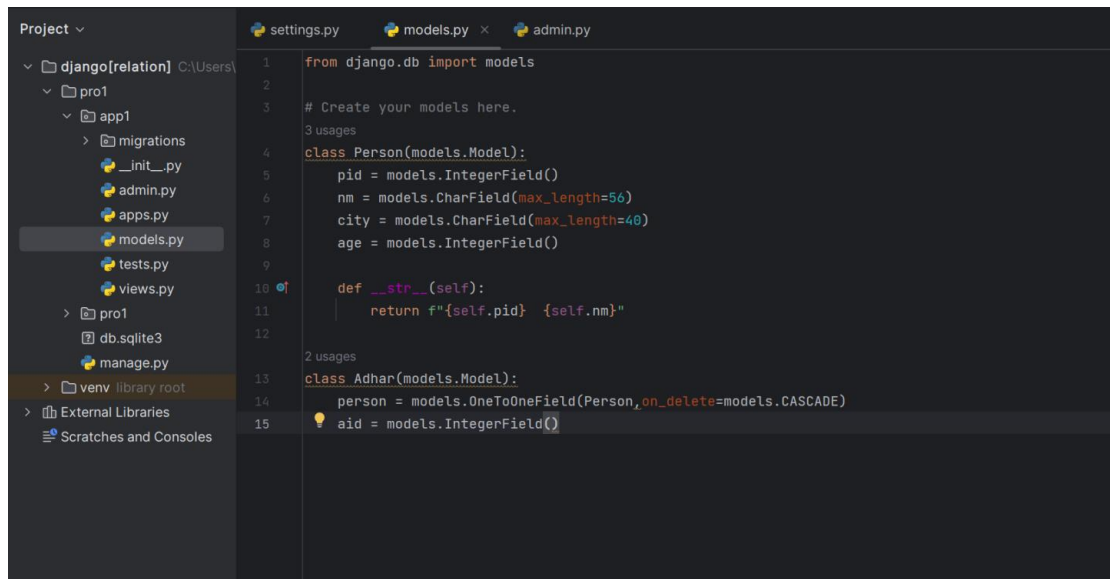
A **one-to-one** relationship means that a record in one table relates to a single record in another table.

- To define a one-to-one relationship, use **OneToOneField**.
- Generally we keep FK field name in lowercase of parent Model.

Let's create an example with a **Person** model and an associated **Adhar** model. Each person will have only one Adhar number.

Step 1: Let's create the **Adhar** model with a one-to-one relationship to the **Person** model:

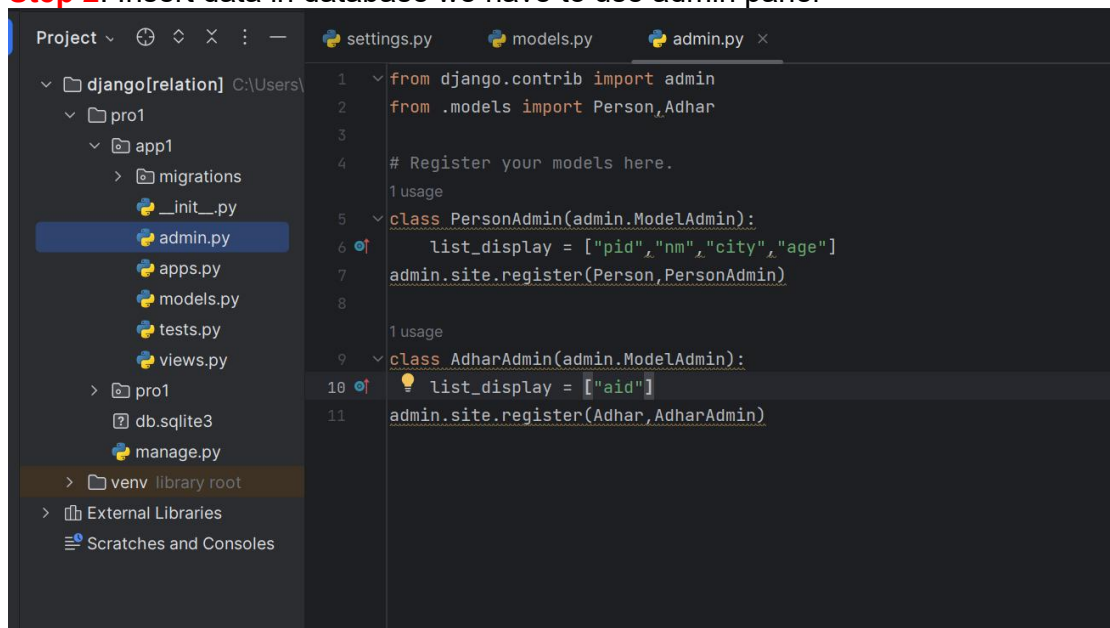
models.py file



```
1 from django.db import models
2
3 # Create your models here.
4 class Person(models.Model):
5     pid = models.IntegerField()
6     nm = models.CharField(max_length=56)
7     city = models.CharField(max_length=40)
8     age = models.IntegerField()
9
10     def __str__(self):
11         return f"{self.pid} {self.nm}"
12
13 class Adhar(models.Model):
14     person = models.OneToOneField(Person, on_delete=models.CASCADE)
15     aid = models.IntegerField()
```

In this example, the **Adhar** model has a **OneToOneField** named **person**, which links it to the **Person** model. The **on_delete=models.CASCADE** argument specifies that if a **Person** instance is deleted, the associated **Adhar** instance will also be deleted.

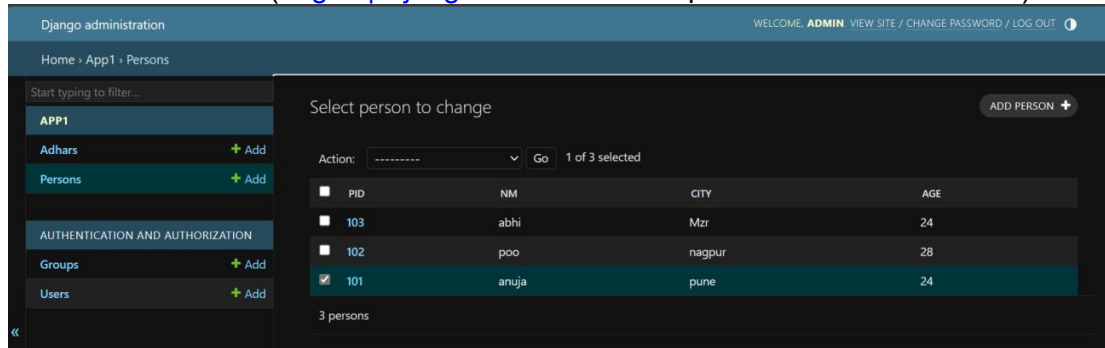
Step 2: Insert data in database we have to use admin panel



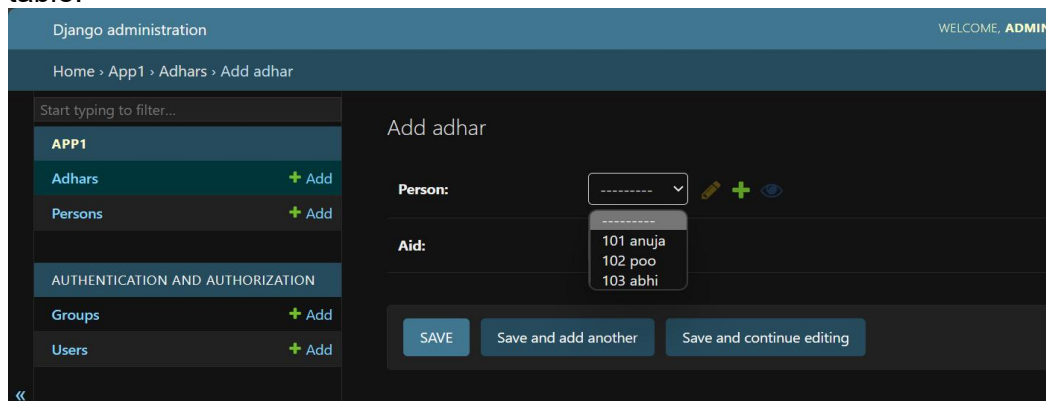
```
1 from django.contrib import admin
2 from .models import Person, Adhar
3
4 # Register your models here.
5 class PersonAdmin(admin.ModelAdmin):
6     list_display = ["pid", "nm", "city", "age"]
7     admin.site.register(Person, PersonAdmin)
8
9 class AdharAdmin(admin.ModelAdmin):
10     list_display = ["aid"]
11     admin.site.register(Adhar, AdharAdmin)
```

Step 3: Command

1. `py manage.py makemigrations(migration)`
2. `py manage.py migrate`
3. `py manage.py createsuperuser(create superuser)`
4. `py manage.py runserver(server run )`
5. enter admin url ([Log in | Django site admin](http://127.0.0.1:8000/admin/) → `http://127.0.0.1:8000/admin/`)



3 Record added in person table we refer 'person' table to add record in 'adhar' table.



One-to-Many Relationship

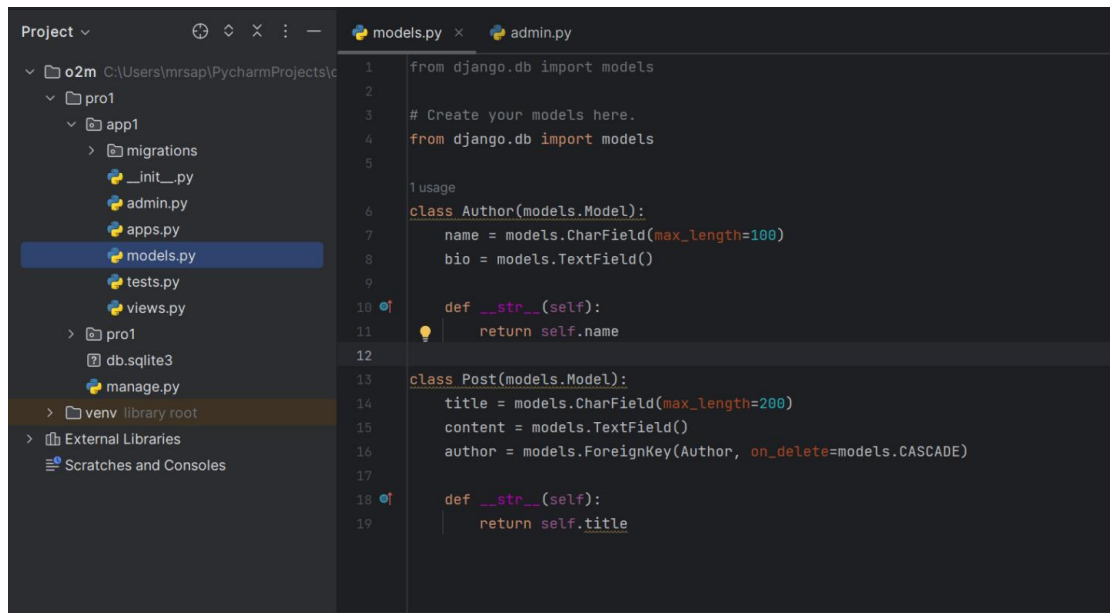
Django, a one-to-many relationship is established between two models when one instance of a model (the "parent" or "one" side) can be related to multiple instances of another model (the "child" or "many" side).

- record in one table relates to multiple records in another table.
- To define a *one-to-many* relationship, use **ForeignKey**.

Let's take an example of a simple blogging platform to illustrate this relationship. We'll have two models: Author and Post. Each author can have multiple posts, creating a one-to-many relationship between authors and posts.

Step 1: Let's create the **Author** model with a one-to-many relationship to the **Post** model using field **ForeignKey**:

models.py file



Author model: Represents an author with a name and bio.

Post model: Represents a blog post with a title, content, and a ForeignKey to Author. In the Post model, author field is defined as a ForeignKey to Author. This indicates a many-to-one relationship, where each post is associated with a single author.

key points

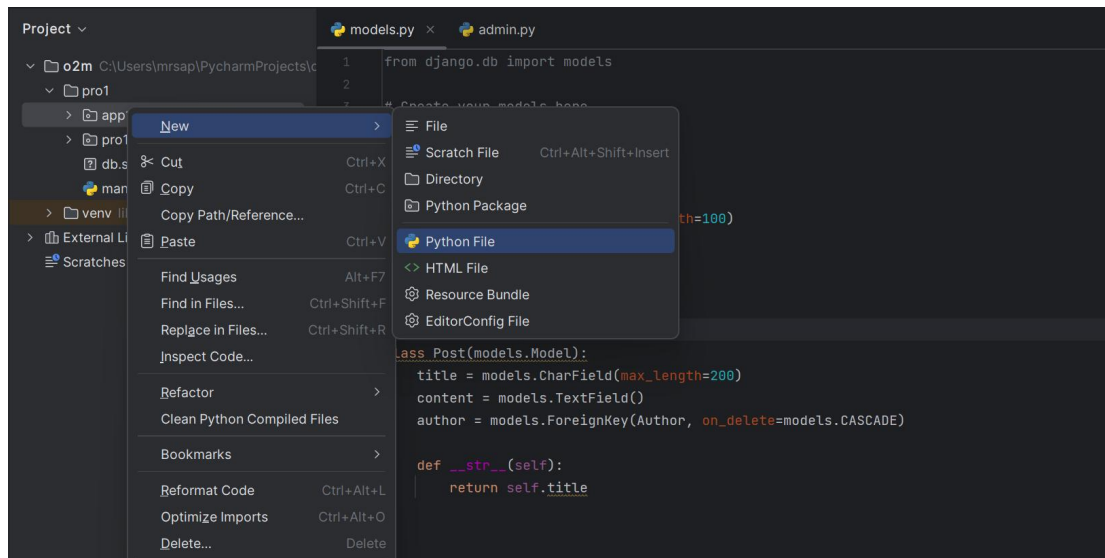
One Author, Many Posts: An instance of the Author model can be associated with multiple instances of the Post model through the ForeignKey.

Step 2: you need to tell Django that you're going to use them by registering the application name in the **INSTALLED_APPS** list in the **settings.py** of the project:(Register app into the INSTALLED_APPS inside settings.py file.

Step 3: Migration Command

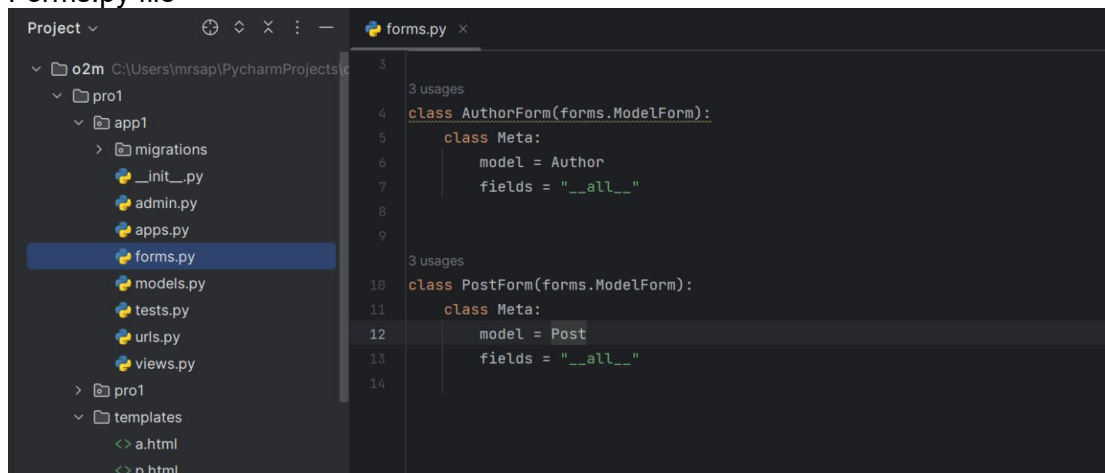
```
py manage.py makemigrations
py manage.py migrate
```

Step 4: Create forms.py file in app directory



Step 5: Next, create Django forms for entering data entry for Author and Post: AuthorForm allows entering author details, while PostForm includes a dropdown to select an author when creating a new post.

Forms.py file



Step 6: Create templates directory inside that directory we have to create new .html file and Register our templates folder in setting.py inside the TEMPLATES list .So that Django will search for html files inside templates directory

```
57
58 TEMPLATES = [
59     {
60         "BACKEND": "django.template.backends.django.DjangoTemplates",
61         "DIRS": ['templates'],
62         "APP_DIRS": True,
63         "OPTIONS": {
64             "context_processors": [
65                 "django.template.context_processors.debug",
66                 "django.template.context_processors.request",
67                 "django.contrib.auth.context_processors.auth",
68                 "django.contrib.messages.context_processors.messages",
69             ],
70         },
71     },
72 ]
73
```

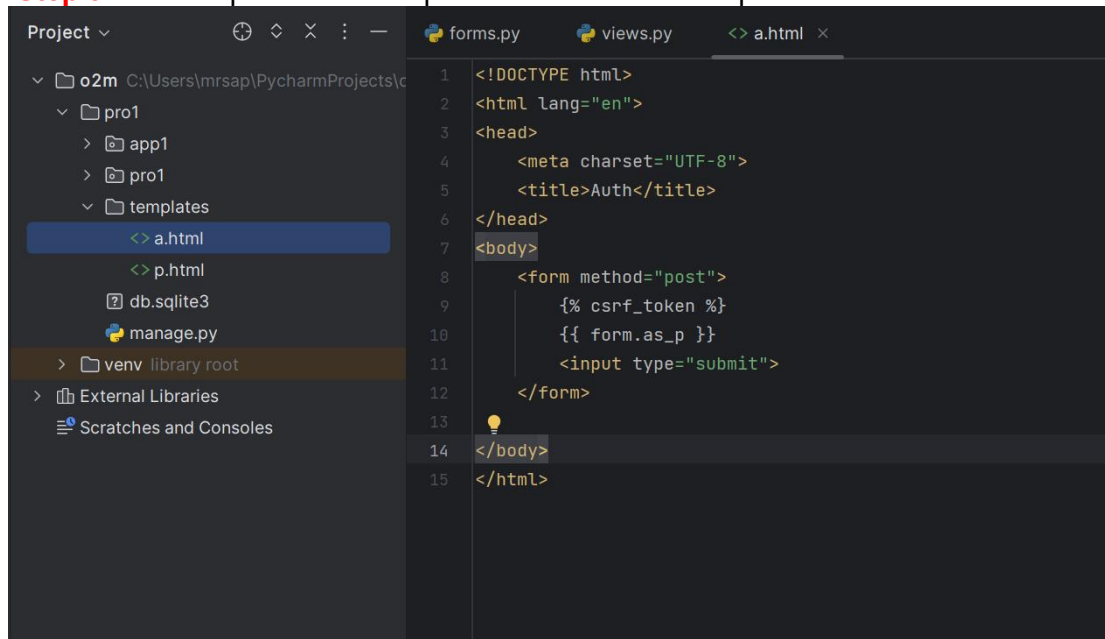
Step 7: Define view for Author and Post in views.py file

```
1 from django.shortcuts import render, redirect
2 from .forms import AuthorForm, PostForm
3
4 # Create your views here.
5
6 def Auth_view(request):
7     form = AuthorForm()
8     if request.method == "POST":
9         form = AuthorForm(request.POST)
10        if form.is_valid():
11            form.save()
12            return redirect("post_url")
13        return render(request, "a.html", {"form": form})
14
15 def Post_view(request):
16     form = PostForm()
17     if request.method == "POST":
18         form = PostForm(request.POST)
19         if form.is_valid():
20             form.save()
21         return render(request, "p.html", {"form": form})
22
```

Step 8: Define url in urls.py file

```
1 from django.urls import path
2 from .views import Auth_view, Post_view
3
4
5 urlpatterns = [
6     path("av/", Auth_view, name="auth_url"),
7     path("pv/", Post_view, name="post_url"),
8 ]
```

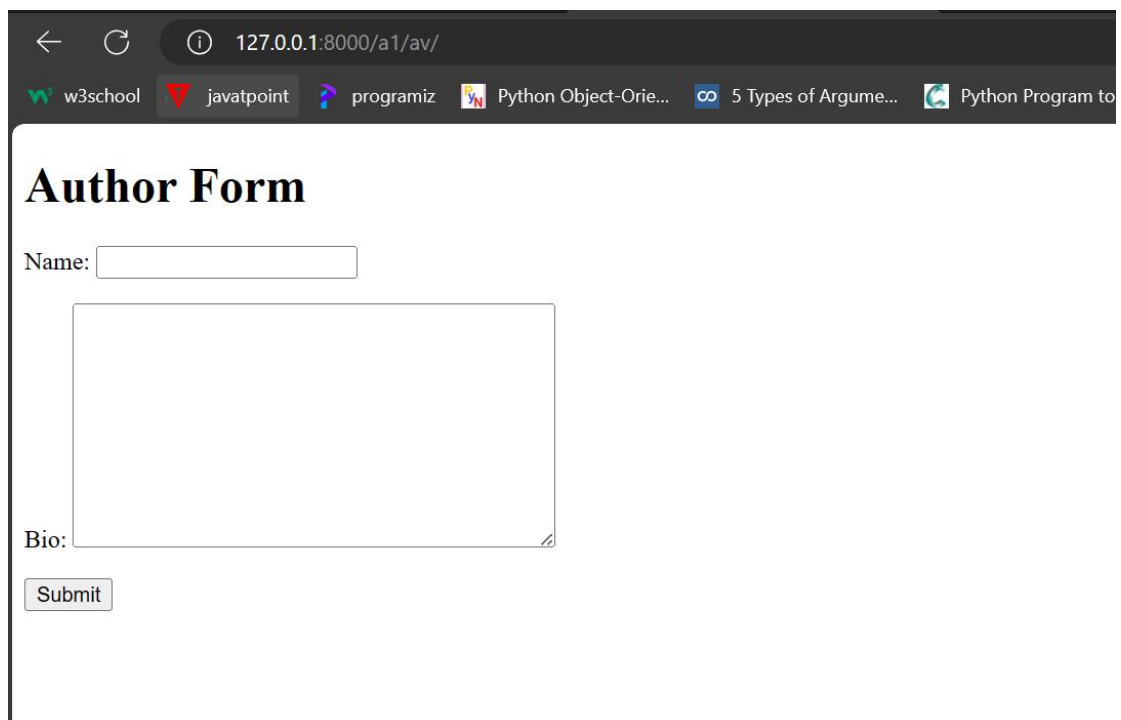
Step 9: Next step to define templates for auth.html and post.html.



The screenshot shows the PyCharm IDE with the 'a.html' file open. The file contains the following HTML code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Auth</title>
6 </head>
7 <body>
8     <form method="post">
9         {% csrf_token %}
10        {{ form.as_p }}
11        <input type="submit">
12    </form>
13
14 </body>
15 </html>
```

Step 10: Run server
Py managr .py runserver



The screenshot shows a web browser at the address 127.0.0.1:8000/a1/av/. The page displays an 'Author Form' with the following fields:

- Name:
- Bio:
- Submit button

Post Form

127.0.0.1:8000/a1/pv/

w3school javatpoint programiz Python Object-Orie... 5 Types of Argume... Python Program to... Why Python

Post Form

Title:

Content:

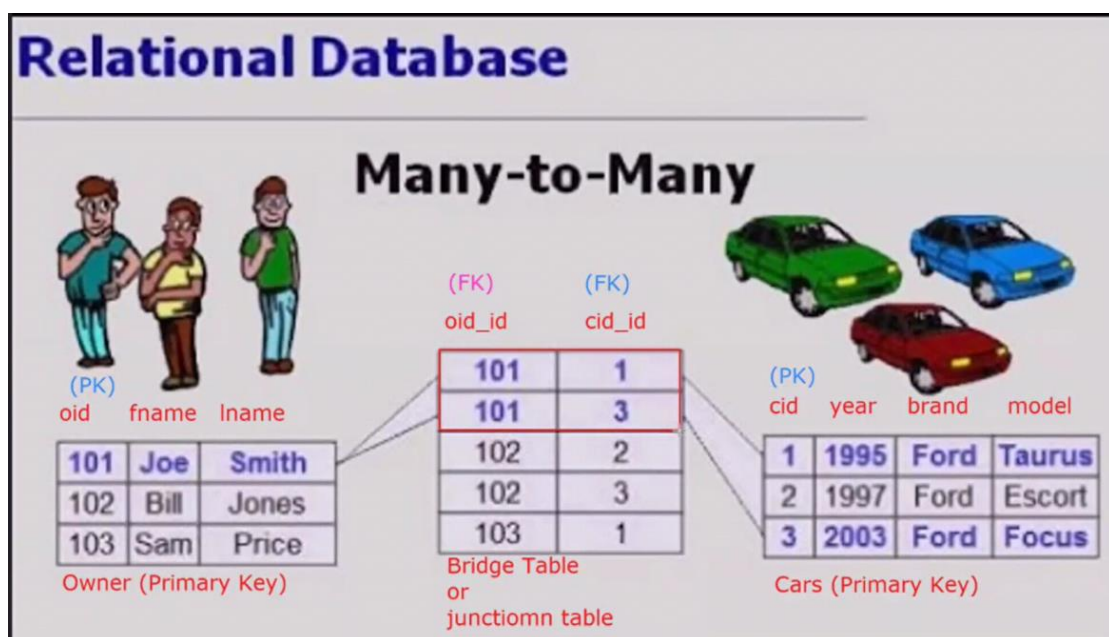
Author:

Many-to-Many Relationship

In Django, a many-to-many relationship between two models, such as an "Owner" and "Car," implies that each owner can possess multiple cars, and each car can belong to multiple

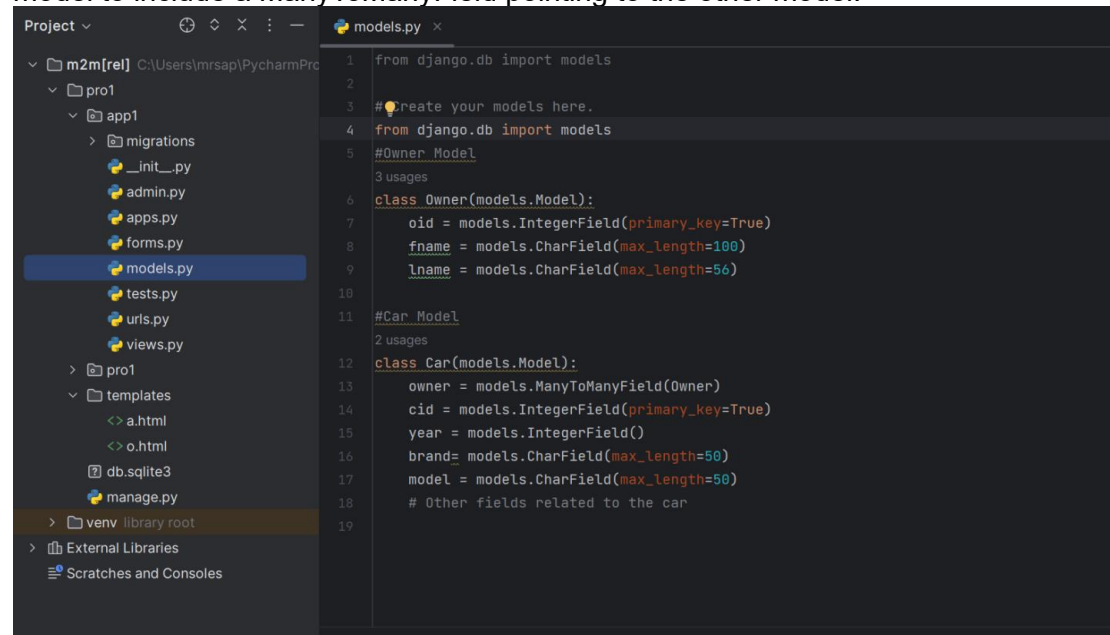
owners. To implement this, you'd use a `ManyToManyField` which creates an intermediate table to manage the relationship between the two models.

- In a **many-to-many** relationship, multiple records in one table relate to many records in another table.
- To define a many-to-many relationship, use **ManyToManyField**.



Step 1: Establishing the Many-to-Many Relationship:

To create the many-to-many relationship between Owner and Car, modify either model to include a ManyToManyField pointing to the other model.



```
1 from django.db import models
2
3 # Create your models here.
4 from django.db import models
5 # Owner Model
6 3 usages
7 class Owner(models.Model):
8     oid = models.IntegerField(primary_key=True)
9     fname = models.CharField(max_length=100)
10    lname = models.CharField(max_length=50)
11
12 # Car Model
13 2 usages
14 class Car(models.Model):
15     owner = models.ManyToManyField(Owner)
16     cid = models.IntegerField(primary_key=True)
17     year = models.IntegerField()
18     brand = models.CharField(max_length=50)
19     model = models.CharField(max_length=50)
20     # Other fields related to the car
```

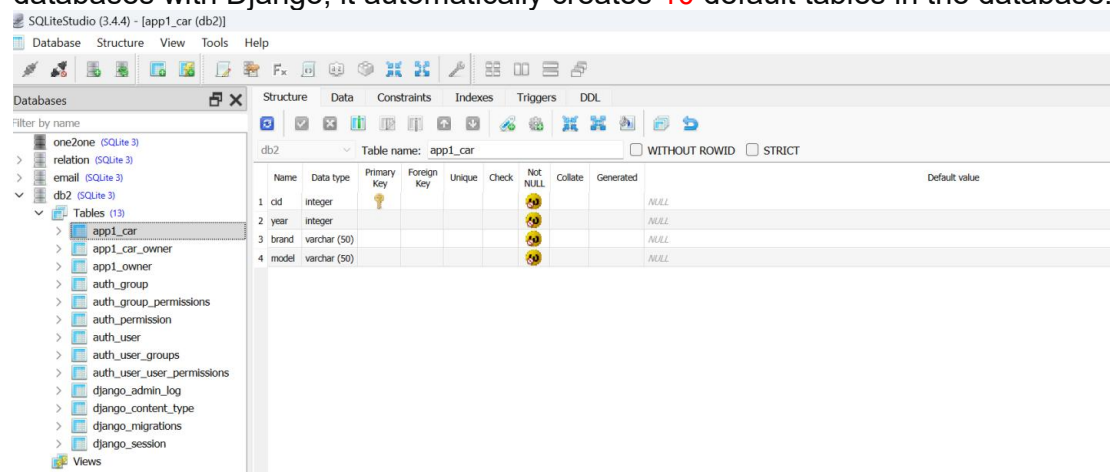
Step 2: Applying Changes and Migrations:

After making these changes to your models, run the following commands in your terminal to create and apply the migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Note:

In many-to-many relationship django automatically create a junction table of **Car** and **Owner** model by the name **app_nm_Car_Owner**. When working with databases with Django, it automatically creates **10** default tables in the database.



Django REST Framework (DRF)

Introduction

These notes are designed for learning Backend Development using Django REST Framework (DRF) with Postman.

This roadmap covers:

- API Basics
- DRF Setup
- Serializers
- CRUD APIs
- Authentication
- JWT Tokens
- Permissions
- Pagination
- Filtering
- Search
- File Upload APIs
- User-wise APIs
- Advanced DRF Concepts

All examples are backend-only and tested using Postman.

1. What is an API?

API stands for Application Programming Interface.

An API allows communication between:

- Frontend and Backend
- Mobile App and Server
- Different Applications

Example:

Frontend sends request:

```
{  
  "name": "Manish"  
}
```

Backend sends response:

```
{  
  "message": "Data Saved"  
}
```

2. What is JSON?

JSON stands for JavaScript Object Notation.

It is used to transfer data between client and server.

Example:

```
{  
  "name": "Rahul",  
}
```

```
"email": "rahul@gmail.com"
}
```

3. Install Django REST Framework

Install Django

```
pip install "django>=5,<6"
```

Install DRF

```
pip install djangorestframework
```

4. Create Project

```
django-admin startproject proj1 .
```

5. Create App

```
python manage.py startapp app1
```

6. Register Apps

settings.py

```
INSTALLED_APPS = [
    'rest_framework',
    'app1',
]
```

7. Create Model

models.py

```
from django.db import models
```

```
class Employee(models.Model):
```

```
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    city = models.CharField(max_length=100)
    salary = models.IntegerField()
```

```
    def __str__(self):
        return self.name
```

8. Run Migrations

```
python manage.py makemigrations
python manage.py migrate
```

9. What is Serializer?

Serializer converts:

- Model Data → JSON
- JSON Data → Model Data

Serializer in DRF works similar to forms.py in Django.

10. Create Serializer

serializers.py

```
from rest_framework import serializers
from .models import Employee

class EmployeeSerializer(serializers.ModelSerializer):

    class Meta:
        model = Employee
        fields = '__all__'
```

11. Function Based API View

views.py

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Employee
from .serializers import EmployeeSerializer

@api_view(['GET'])
def employee_list(request):

    data = Employee.objects.all()

    serializer = EmployeeSerializer(data, many=True)

    return Response(serializer.data)
```

12. Add URL

urls.py

```
from django.urls import path
from .views import *

urlpatterns = [
    path('employee/', employee_list),
]
```

13. Run Server

`python manage.py runserver`

API URL:

`http://127.0.0.1:8000/employee/`

14. What is Postman?

Postman is a tool used to test APIs.

Using Postman you can:

- Send GET request
- Send POST request
- Send PUT request
- Send DELETE request
- Send JSON data

Download:

<https://www.postman.com/downloads/>

15. GET API

Used to fetch data.

Example:

GET

`http://127.0.0.1:8000/employee/`

16. POST API

Used to save data.

views.py

```
@api_view(['POST'])
def employee_create(request):

    serializer = EmployeeSerializer(data=request.data)

    if serializer.is_valid():
        serializer.save()

    return Response(serializer.data)

return Response(serializer.errors)
```

17. POST Request Body

Postman → Body → raw → JSON

```
{
  "name": "Manish",
  "email": "manish@gmail.com",
  "city": "Ahmedabad",
  "salary": 50000
}
```

18. PUT API

Used to update data.

```
@api_view(['PUT'])
def employee_update(request, pk):

    obj = Employee.objects.get(id=pk)

    serializer = EmployeeSerializer(obj, data=request.data)

    if serializer.is_valid():
```

```
serializer.save()

return Response(serializer.data)

return Response(serializer.errors)
```

19. DELETE API

Used to delete data.

```
@api_view(['DELETE'])
def employee_delete(request, pk):

    obj = Employee.objects.get(id=pk)

    obj.delete()

    return Response({'message': 'Deleted'})
```

20. Complete CRUD Operations

Method	Purpose
GET	Fetch data
POST	Create data
PUT	Update data
DELETE	Delete data

21. APIView

APIView provides class-based API views.

Example:

```
from rest_framework.views import APIView
from rest_framework.response import Response

class EmployeeAPIView(APIView):

    def get(self, request):
        pass
```



```
def post(self, request):  
    pass
```

22. Status Codes

```
from rest_framework import status  
Common Status Codes:
```

```
status.HTTP_200_OK  
status.HTTP_201_CREATED  
status.HTTP_400_BAD_REQUEST  
status.HTTP_404_NOT_FOUND
```

23. Validation in Serializer

```
class EmployeeSerializer(serializers.ModelSerializer):
```

```
    class Meta:  
        model = Employee  
        fields = '__all__'
```

```
    def validate_salary(self, value):
```

```
        if value < 1000:  
            raise serializers.ValidationError('Salary too low')
```

```
        return value
```

24. Authentication

Authentication verifies user identity.

Types:

- Session Authentication
 - Token Authentication
 - JWT Authentication
-

25. JWT Authentication

JWT = JSON Web Token

Used for secure APIs.

Install:

```
pip install djangorestframework-simplejwt
```

settings.py

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': (  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    )  
}
```

JWT URLs

```
from rest_framework_simplejwt.views import (  
    TokenObtainPairView,  
    TokenRefreshView,  
)  
  
urlpatterns = [  
    path('api/token/', TokenObtainPairView.as_view()),  
    path('api/token/refresh/', TokenRefreshView.as_view()),  
]
```

26. Login API

POST Request:

```
{  
  "username": "admin",  
  "password": "admin123"  
}
```

Response:

```
{  
  "refresh": "token",  
  "access": "token"  
}
```

27. Access Protected APIs

In Postman:

Authorization → Bearer Token

Paste:

access_token

28. Permissions

Permissions control API access.

Example:

```
from rest_framework.permissions import IsAuthenticated  
permission_classes = [IsAuthenticated]
```

29. User-wise Data

Show only logged-in user data.

Employee.objects.filter(user=request.user)

30. Pagination

Pagination splits data into pages.

settings.py

```
REST_FRAMEWORK = {  
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPaginatio  
n',  
    'PAGE_SIZE': 5  
}
```

31. Search Filter

Install:

pip install django-filter

settings.py

```
INSTALLED_APPS = [  
    'django_filters',  
]
```

views.py

```
from rest_framework import filters
```

```
filter_backends = [filters.SearchFilter]
```

```
search_fields = ['name', 'email']
```

Example:

http://127.0.0.1:8000/employee/?search=manish

32. Ordering Filter

```
from rest_framework import filters
```

```
filter_backends = [filters.OrderingFilter]
```

```
ordering_fields = ['salary']
```

Example:

```
http://127.0.0.1:8000/employee/?ordering=salary
```

33. File Upload API

models.py

```
image = models.ImageField(upload_to='employee_images/')
```

Install Pillow:

```
pip install pillow
```

34. ViewSets

ViewSets reduce repeated CRUD code.

```
from rest_framework.viewsets import ModelViewSet
```

```
class EmployeeViewSet(ModelViewSet):
```

```
    queryset = Employee.objects.all()
```

```
    serializer_class = EmployeeSerializer
```

35. Routers

```
from rest_framework.routers import DefaultRouter

router = DefaultRouter()

router.register('employee', EmployeeViewSet)

urlpatterns = router.urls
```

36. API Testing Flow

Step 1 → Create Model
Step 2 → Create Serializer
Step 3 → Create API View
Step 4 → Add URL
Step 5 → Run Server
Step 6 → Open Postman
Step 7 → Test GET API
Step 8 → Test POST API
Step 9 → Test PUT API
Step 10 → Test DELETE API

37. Important Interview Questions

Q1. What is DRF?

Django REST Framework used to build APIs.

Q2. What is Serializer?

Converts model data into JSON.

Q3. Difference between Django and DRF?

Django builds web apps. DRF builds APIs.

Q4. What is JWT?

Token-based authentication system.

Q5. What is APIView?

Class-based API view.

Q6. What is ModelViewSet?

Automatically handles CRUD APIs.

Q7. What is Postman?

API testing tool.

38. Complete DRF Learning Roadmap

Django Basics ↓ Models ↓ Serializers ↓ Function Based APIs ↓ APIView ↓ CRUD APIs ↓ Postman ↓ JWT Authentication ↓ Permissions ↓ Pagination ↓ Filtering ↓ Search ↓ ViewSets ↓ Routers ↓ File Upload APIs ↓ Deployment

39. Final Summary

Django REST Framework is used to build secure and scalable backend APIs.

Important Topics:

- APIs
- JSON
- Serializers
- CRUD APIs

- JWT Authentication
- Permissions
- Pagination
- Filtering
- Postman Testing
- ViewSets
- Routers

These concepts are highly important for internships and backend developer jobs.

40. Serializer in Django REST

Framework (DRF)

What is a Serializer?

A Serializer in Django REST Framework is used to convert:

- Django Model Data → JSON
- JSON Data → Python Objects

Serializer acts like a bridge between:

- Django Backend
- API JSON Response

Serializer works similarly to forms.py in Django, but it is used for APIs.

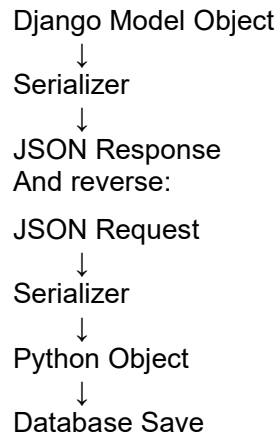
Why Serializer is Needed

Django models and QuerySets cannot be directly returned in API responses.

APIs understand JSON format.

So serializers convert Python/Django objects into JSON.

Serializer Conversion Flow



Example Model

models.py

```
from django.db import models

class Employee(models.Model):

    name = models.CharField(max_length=100)

    email = models.EmailField(unique=True)

    city = models.CharField(max_length=100)

    salary = models.IntegerField()
```

Example JSON Output

```
{
  "id": 1,
```

```
"name": "Manish",  
"email": "manish@gmail.com",  
"city": "Ahmedabad",  
"salary": 50000  
}
```

Types of Serializers

There are mainly 2 types:

Serializer Type	Purpose
Serializer	Manual serializer
ModelSerializer	Automatic model-based serializer

1. Serializer (Manual Serializer)

In this serializer, all fields are manually defined.

serializers.py

```
from rest_framework import serializers  
  
class EmployeeSerializer(serializers.Serializer):  
    name = serializers.CharField(max_length=100)  
    email = serializers.EmailField()  
    city = serializers.CharField(max_length=100)  
    salary = serializers.IntegerField()
```

Advantages of Serializer

- Full control over fields

- Custom logic
 - No model dependency
-

Disadvantages

- More code
 - Manual field creation
 - Less reusable
-

2. ModelSerializer (Most Important)

ModelSerializer automatically creates serializer fields from Django models.
This is MOST commonly used in real projects.

serializers.py

```
from rest_framework import serializers
from .models import Employee

class EmployeeSerializer(serializers.ModelSerializer):

    class Meta:
        model = Employee
        fields = '__all__'
```

Benefits of ModelSerializer

- Less code
- Automatic field generation
- Easy CRUD APIs
- Built-in validation

What is class Meta?

Meta class is used for serializer configuration.

It tells DRF:

- Which model to use
- Which fields to include
- Which fields to exclude

model = Employee

model = Employee

Meaning:

Use Employee model fields inside serializer.

fields = 'all'

fields = '__all__'

Meaning:

Include all model fields.

Specific Fields

fields = ['name', 'email']

Only selected fields will be included.

exclude

```
exclude = ['salary']
```

All fields except salary will be included.

Important Rule

Do NOT use both together:

```
fields = '__all__'
```

```
exclude = ['salary']
```

This will raise an error.

Use only one:

- fields
 - or exclude
-

Serialization Process

Serialization means:

Python Object → JSON

Example:

```
obj = Employee.objects.get(id=1)
```

```
serializer = EmployeeSerializer(obj)
```

```
print(serializer.data)
```

Output:

```
{
  "id": 1,
  "name": "Manish",
  "email": "manish@gmail.com"
}
```

Deserialization Process

Deserialization means:

JSON → Python Object

Example:

```
serializer = EmployeeSerializer(data=request.data)
```

Here:

- request.data contains JSON data from frontend/Postman.
-

Validation in Serializer

```
if serializer.is_valid():
```

Serializer checks:

- Required fields
 - Empty fields
 - Data types
 - Email format
 - Unique fields
-

Save Data

```
serializer.save()
```

This saves validated data into database.

serializer.data

Returns serialized JSON data.

Example:

```
serializer.data
```

Output:

```
{  
  "id": 1,  
  "name": "Rahul"  
}
```

serializer.errors

Returns validation errors.

Example:

serializer.errors

Output:

```
{  
  "email": [  
    "This field is required."  
  ]  
}
```

serializer.is_valid()

Checks whether incoming JSON data is valid.

Example:

serializer.is_valid()

Returns:

- True
 - False
-

Complete POST API Example

views.py

```
from rest_framework.decorators import api_view  
from rest_framework.response import Response
```

```
from .serializers import EmployeeSerializer

@api_view(['POST'])
def employee_create(request):

    serializer = EmployeeSerializer(data=request.data)

    if serializer.is_valid():

        serializer.save()

        return Response(serializer.data)

    return Response(serializer.errors)
```

POST API Flow

Step 1 → Postman sends JSON data

```
{
  "name": "Manish",
  "email": "manish@gmail.com"
}
```

Step 2 → request.data receives JSON

Step 3 → Serializer validates data

```
serializer.is_valid()
```

Step 4 → Save data

```
serializer.save()
```

Step 5 → Return JSON response

```
return Response(serializer.data)
```

Custom Validation

```
from rest_framework import serializers
```

```
class EmployeeSerializer(serializers.ModelSerializer):
```

```
    class Meta:
```

```
        model = Employee
```

```
        fields = '__all__'
```

```
    def validate_salary(self, value):
```



```
if value < 1000:
    raise serializers.ValidationError('Salary too low')

return value
```

Advantages of Serializer

Advantage	Description
Converts Model to JSON	API response
Converts JSON to Model	Save data
Validation Support	Automatic validation
Cleaner Code	Less manual work
Secure Data Handling	Control fields

Difference Between Django Forms and

DRF Serializers

Django Forms	DRF Serializers
Used in HTML forms	Used in APIs
Works with Templates	Works with JSON
Frontend forms	Backend APIs

Real Project Usage

Serializers are used in:

- Login APIs
- Signup APIs
- Product APIs
- Employee APIs
- Mobile App APIs
- React + Django APIs

Most Important Serializer Methods

Method	Purpose
<code>serializer.data</code>	Return JSON data
<code>serializer.is_valid()</code>	Validate data
<code>serializer.save()</code>	Save data
<code>serializer.errors</code>	Show errors

Final Summary

Serializer is one of the most important components in Django REST Framework.

It is used to:

- Convert Django models into JSON
- Convert JSON into Python objects
- Validate API data
- Save data into database
- Return API responses

Without serializers, DRF APIs cannot properly communicate using JSON.